



**UNIVERSITÀ
DEGLI STUDI
DI BERGAMO**

Dipartimento di
Ingegneria Gestionale, dell'Informazione e della Produzione

Corso di Laurea in
Ingegneria Informatica

Progettazione algoritmi e computabilità

Giorgio Passarella

Matricola n. 1079287

Davide Brambilla

Matricola n. 1080752

Massimiliano Federico Gervasoni

Matricola n. 1069211

Anno Accademico

2025/2026

Indice

Elenco delle figure	iii
1 Iterazione 0	1
1.1 Introduzione	1
1.2 Requisiti	3
1.3 Casi D'uso	5
1.4 Deployment Diagram	7
1.5 Architettura del Sistema	8
1.6 Strumenti utilizzati	11
2 Iterazione 1	15
2.1 Casi d'uso implementati	15
2.2 Diagramma delle Interfacce	19
2.3 UML Class Diagram	20
2.4 Component Diagram	21
2.5 Deployment Diagram	23
2.6 Ottimizzazione del percorso	25
2.7 Test	27
2.7.1 Test: CodeMR	27
2.7.2 Test: Junit	29
2.7.3 API Esposte	32
3 Iterazione 2	37
3.1 Casi d'uso implementati	37
3.2 Diagramma delle interfacce	42
3.3 UML Class Diagram	44
3.4 Component Diagram	46
3.5 Deployment diagram	47
3.6 Test	48
3.6.1 Test: CodeMR	48

3.6.2	Test: Junit	51
3.6.3	API Esposte	54

Elenco delle figure

1.1	Casi d'uso	5
1.2	Diagramma di deployment	7
1.3	Architettura MVC del Sistema	9
2.1	Diagramma delle interfacce	19
2.2	Diagramma delle classi	20
2.3	Diagramma dei componenti	21
2.4	Diagramma di deployment	23
2.5	Analisi della complessità.	27
2.6	Grado di accoppiamento.	27
2.7	Livello di coesione.	27
2.8	Distribuzione dei componenti: organizzazione delle classi nei package Model, View/DTO, Service e Controller.	28
2.9	Test degli endpoint di autenticazione.	29
2.10	Test della logica di autenticazione.	29
2.11	Test degli endpoint profilo utente.	29
2.12	Test della logica gestione utente.	30
2.13	Test degli endpoint monumenti.	30
2.14	Test integrazione Overpass API.	30
2.15	Test creazione viaggi con Nominatim.	30
2.16	Test degli endpoint relativi ai viaggi.	30
2.17	Validazione approfondita dell'algoritmo di ottimizzazione TSP.	31
2.18	Test del servizio di ottimizzazione.	31
2.19	Test su Postman: Registrazione di un nuovo utente nel sistema.	32
2.20	Test su Postman: Login utente e generazione del token di autenticazione JWT.	33
2.21	Test su Postman: Recupero della lista dei monumenti per la città di Bergamo.	34
2.22	Test su Postman: Payload di richiesta per la creazione di un nuovo viaggio.	34
2.23	Test su Postman: Risposta del server con il salvataggio del viaggio.	35

2.24	Test su Postman: Esecuzione dell'algoritmo di ottimizzazione TSP. La risposta mostra l'itinerario riordinato con calcolo di distanza e tempi totali.	35
2.25	Test su Postman: Eliminazione corretta di un viaggio con risposta 204 No Content.	36
3.1	Diagramma delle interfacce - Iterazione 2.	42
3.2	UML Class Diagram - Iterazione 2.	44
3.3	Diagramma dei componenti	46
3.4	Complessità.	49
3.5	Accoppiamento.	49
3.6	Lack of Cohesion.	49
3.7	Dettaglio della classe TripService, identificata con complessità alta.	50
3.8	Distribuzione delle metriche all'interno dei package architetturali.	50
3.9	Test del filtro di autenticazione JWT e gestione dei token.	51
3.10	Validazione del Provider JWT.	51
3.11	Test del Custom User Details Service.	51
3.12	Espansione dei test sugli endpoint del TripController.	52
3.13	Espansione dei test della logica di business nel TripServiceTest.	53
3.14	Test su Postman: Payload di richiesta per l'aggiornamento di un viaggio.	54
3.15	Test su Postman: Risposta del server dopo l'aggiornamento del viaggio.	55

Capitolo 1

Iterazione 0

1.1 Introduzione

OptiTour è un'app che ottimizza gli itinerari turistici calcolando automaticamente il percorso più efficiente per visitare una serie di luoghi nel tempo a disposizione.

Scegliere cosa visitare è semplice, ma organizzare le tappe nell'ordine migliore richiede tempo, esperienza e una valutazione accurata dei tempi di spostamento e di visita. A differenza delle comuni app di mappe, OptiTour non si limita a mostrare i punti di interesse, ma trasforma una lista di monumenti in un itinerario ottimale.

L'utente seleziona il punto di partenza e i luoghi che desidera visitare, indicando quanto tempo ha a disposizione. L'app, elaborando queste informazioni, dà origine al percorso più rapido, garantendo che l'itinerario rispetti la finestra temporale impostata.

Con OptiTour, l'utente può:

- **Pianificazione Itinerari Smart:** selezione dei punti di interesse (POI) e calcolo del ciclo ottimale per ridurre chilometri e tempi di percorrenza.
- **Gestione Flessibile:** possibilità di creare nuovi viaggi (manuali o casuali), modificare le tappe, visualizzare dettagli o eliminare percorsi.
- **Social e Storico:** salvataggio dei percorsi preferiti, consultazione dello storico dei viaggi effettuati, condivisione e aggiunta di recensioni.
- **Gestione Profilo:** registrazione, login e gestione sicura dei dati personali e delle credenziali.

L'applicazione si interfaccia direttamente con provider di servizi geografici esterni per garantire che il calcolo delle distanze e dei tempi sia basato su dati reali e aggiornati.

Perché scegliere OptiTour?

OptiTour nasce per rispondere a un'esigenza specifica: l'efficienza nella programmazione di itinerari. Spesso i turisti perdono tempo prezioso camminando avanti e indietro per la città a causa di una pianificazione non ottimale o assente. Questo problema, noto come *Traveling Salesman Problem* (TSP), diventa difficile da risolvere mentalmente man mano che il numero di luoghi da visitare aumenta.

OptiTour automatizza questo processo complesso utilizzando algoritmi avanzati sui grafi ed euristiche dedicate. Il sistema non offre solo un percorso, ma il miglior percorso possibile, rendendolo lo strumento ideale per diverse tipologie di utenti: dal turista che vuole massimizzare le visite nel tempo a disposizione, allo sportivo che cerca un percorso di allenamento circolare che tocchi specifici punti di interesse, fino ai professionisti dell'ospitalità che desiderano offrire itinerari personalizzati ai propri ospiti.

Con un'interfaccia intuitiva che separa nettamente la logica di presentazione da quella algoritmica, OptiTour offre una soluzione potente ma accessibile per trasformare il modo in cui esploriamo le città.

1.2 Requisiti

Il sistema deve soddisfare i seguenti requisiti funzionali e non funzionali per garantire un'esperienza di pianificazione ottimale:

Requisiti funzionali

- **Gestione autenticazione e profilo:** gli utenti devono poter effettuare la registrazione, il login e il logout. Il sistema deve permettere la gestione del profilo includendo la modifica dei dati personali, il cambio password e l'eliminazione definitiva dell'account.
- **Visualizzazione e catalogo:** il sistema deve offrire una vista astratta per la visualizzazione dei percorsi, permettendo all'utente di sfogliare un catalogo, applicare filtri di ricerca ed esportare l'itinerario desiderato.
- **Creazione itinerario manuale:** l'utente può generare un nuovo viaggio impostando un punto di partenza e selezionando manualmente i monumenti e le tappe d'interesse.
- **Creazione itinerario casuale:** generazione automatica basata su percorsi esistenti o nuovi, con la possibilità di definire tempistiche specifiche per la visita.
- **Ottimizzazione del percorso (TSP):** il sistema deve integrare un modulo di ottimizzazione che, interfacciandosi con un Provider di servizi geografici esterno, calcoli il percorso ottimo. Tale ottimizzazione è inclusa automaticamente durante la creazione o la modifica delle tappe.
- **Gestione e stato viaggi:** gli utenti devono poter salvare percorsi, modificarne le tappe, eliminarli o contrassegnarli come "completati".
- **Social e storico:** il sistema deve gestire uno storico dei viaggi che permetta la condivisione degli itinerari e l'aggiunta di recensioni per i percorsi effettuati.

Requisiti non funzionali

- **Architettura modulare:** il sistema deve separare nettamente le funzionalità di interfaccia (vista utente) dalle logiche di elaborazione e integrazione esterna (vista di sistema).
- **Usabilità:** l'interfaccia deve permettere l'estensione fluida di funzionalità secondarie (es. filtri, esportazione) senza interrompere il flusso principale di creazione del viaggio.

- **Efficienza e accuratezza:** il calcolo dell'ottimizzazione deve essere tempestivo e basato su dati geografici reali forniti da terze parti.
- **Sicurezza:** garantire l'accesso protetto alle funzionalità di gestione profilo e la persistenza sicura dei percorsi salvati e dello storico.

1.3 Casi D'uso

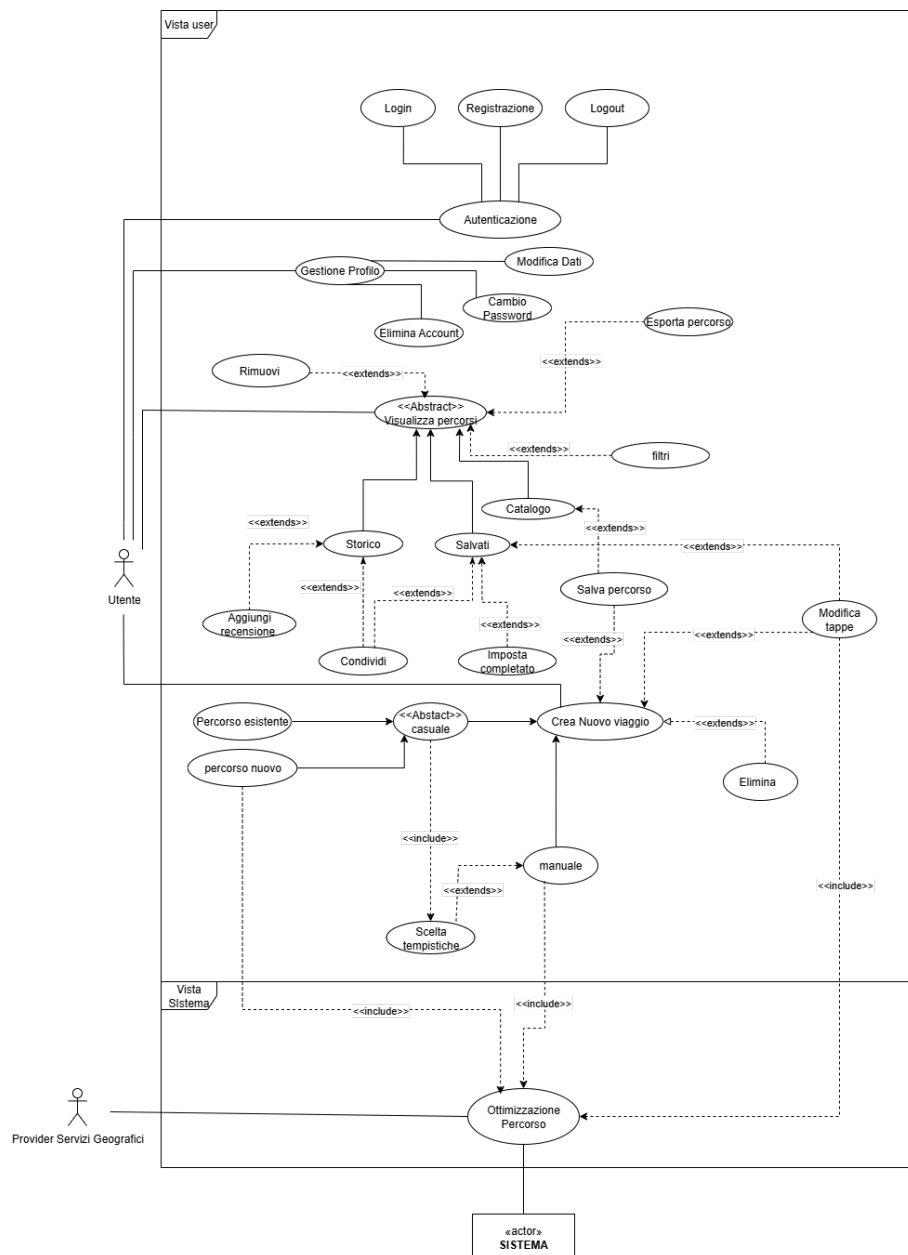


Figura 1.1: Casi d'uso

Di seguito sono riportati i casi d'uso relativi al sistema di gestione percorsi e viaggi.

- **UC1 - Autenticazione**

- **UC1.1 - Registrazione:** un nuovo utente può creare un account personale fornendo i dati richiesti.

- **UC1.2 - Login:** un utente registrato può accedere al proprio account inserendo le proprie credenziali per iniziare una sessione e usare le funzionalità del sistema.
- **UC1.3 - Logout:** l'utente può terminare la propria sessione in modo sicuro.
- **UC2 - Gestione profilo**
 - **UC2.1 - Modifica dati:** l'utente può modificare i propri dati personali in caso di necessità.
 - **UC2.2 - Elimina account:** l'utente può eliminare definitivamente il proprio account.
- **UC3 - Visualizza percorsi:** l'utente può visualizzare i dettagli di un percorso.
 - **UC3.1 - Catalogo:** è possibile visualizzare percorsi creati da altri utenti e inseriti in un catalogo.
 - **UC3.2 - Salvati:** è possibile visualizzare percorsi provenienti dall'elenco dei percorsi salvati.
 - **UC3.3 - Storico:** è possibile visualizzare percorsi provenienti dallo storico dei percorsi effettuati.
- **UC4 - Filtri ricerca:** l'utente può filtrare i percorsi in base a diversi criteri.
- **UC5 - Salva percorso:** l'utente può salvare un percorso nella sezione dei percorsi preferiti.
- **UC6 - Esporta percorso:** l'utente può esportare un percorso in un formato condivisibile.
- **UC7 - Rimuovi percorso salvato:** l'utente può rimuovere un percorso dai percorsi salvati.
- **UC8 - Condividi percorso:** l'utente può condividere un percorso con altri utenti.
- **UC9 - Aggiungi recensione:** l'utente può aggiungere una recensione a un percorso che è stato effettuato.
- **UC10 - Imposta percorso completato:** l'utente può contrassegnare un percorso come completato.
- **UC11 - Imposta punto di partenza:** l'utente seleziona il punto da cui far partire e finire il percorso.

- **UC12 - Crea nuovo viaggio:**
 - **UC12.1 - Manuale:** l'utente può far creare il proprio percorso scegliendo autonomamente le tappe da visitare.
 - **UC12.2 - Casuale:**
 - * **UC12.2.1 - Percorso esistente:** l'utente fa scegliere al sistema in modo casuale un percorso già esistente nel catalogo.
 - * **UC12.2.2 - Percorso nuovo:** l'utente fa generare al sistema in modo casuale un nuovo percorso.
- **UC13 - Scelta tempistiche:** l'utente può definire le tempistiche del viaggio.
- **UC14 - Ottimizzazione percorso:** il sistema ottimizza tramite un algoritmo specifico l'ordine delle tappe.
- **UC15 - Modifica tappe:** l'utente può modificare le tappe di un percorso.
- **UC16 - Elimina tappa:** l'utente può eliminare una tappa dal percorso.

1.4 Deployment Diagram

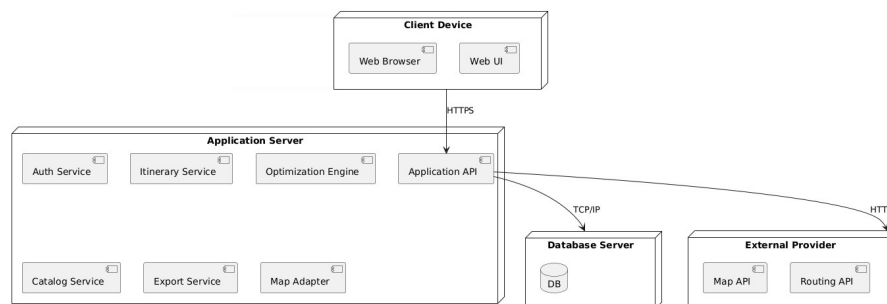


Figura 1.2: Diagramma di deployment

La Figura mostra l'organizzazione del sistema secondo un modello multilivello che separa l'interfaccia utente, i servizi applicativi e le integrazioni esterne.

- **Livello Client.** L'accesso al sistema avviene tramite un browser web. La Web UI comunica con il backend attraverso richieste HTTPS indirizzate all'Application API. Questo livello si occupa esclusivamente della presentazione e dell'interazione con l'utente finale.
- **Application Server.** Costituisce il cuore dell'applicazione e ospita diversi servizi interni:

- **Auth Service:** gestisce autenticazione e autorizzazione;
- **Itinerary Service:** gestisce la creazione e modifica degli itinerari;
- **Optimization Engine:** calcola il percorso ottimale tra i monumenti selezionati;
- **Catalog Service:** fornisce i dati relativi ai monumenti e ai punti di interesse;
- **Export Service:** genera output esportabili;
- **Map Adapter:** gestisce le comunicazioni con i servizi di mappe esterni.

Tutti i servizi comunicano tramite l'Application API, che funge da punto di orchestrazione.

- **Database Server.** Ospita il database principale dell'applicazione e comunica con l'Application API. Gestisce dati persistenti come utenti, monumenti, itinerari e preferenze.
- **External Provider.** Il sistema si integra con servizi esterni per ottenere informazioni geografiche e di navigazione:
 - **Map API:** fornisce mappe, geocodifica e dati geografici;
 - **Routing API:** calcola distanze e tempi di percorrenza, supportando l'algoritmo di ottimizzazione.

1.5 Architettura del Sistema

Il sistema segue il pattern architetturale **Model-View-Controller (MVC)**, una scelta progettuale che permette di separare la logica di presentazione, la logica di ottimizzazione degli itinerari e la gestione dei dati. Tale separazione favorisce la modularità del codice, facilitando la manutenzione e l'estendibilità del software nel tempo.

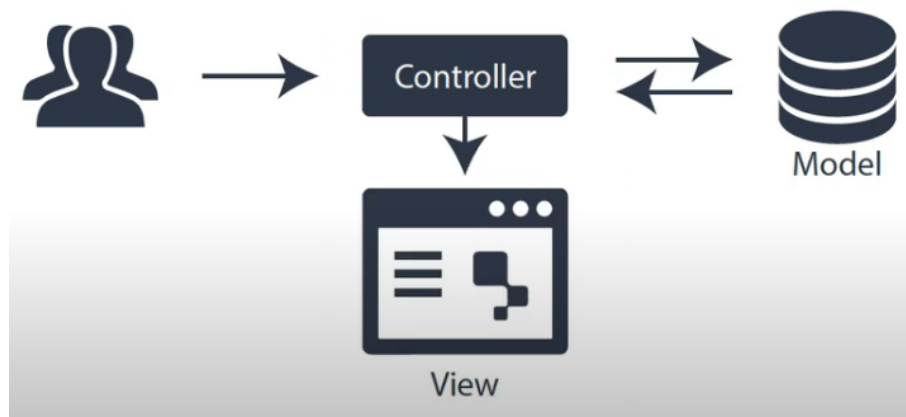


Figura 1.3: Archiettura MVC del Sistema

Model. Costituisce il nucleo logico e strutturale del sistema, responsabile della gestione dei dati, delle regole di dominio e dei meccanismi di persistenza dei dati.

In questo sistema:

- **Database NoSQL:** i dati sono memorizzati in MongoDB. La scelta di un database non relazionale è dettata dalla necessità di gestire con flessibilità i Points of Interest (POI) e i dati geografici. Per la gestione e la visualizzazione dei dati a livello amministrativo, viene utilizzata la GUI MongoDB Compass
- **Backend in Java:** il core del sistema è implementato utilizzando il framework Spring Boot, che orchestra i servizi attraverso un'architettura orientata ai componenti.
- **Integrazione geografica:**
 - **OpenStreetMap:** utilizzato come provider per le coordinate geografiche e i metadati dei POI.
 - **GraphHopper:** integrato come motore di routing per il calcolo del percorso ottimale tra i punti selezionati.
- **Data Access Layer:** l'interazione con il database avviene tramite Spring Data MongoDB.

View. È responsabile dell'interfaccia utente e della visualizzazione dei dati elaborati dal sistema:

- **React.js:** il frontend è sviluppato con la libreria React, utilizzando componenti funzionali e JSX per una UI reattiva. Lo sviluppo avviene tramite Visual Studio Code, ottimizzato per la gestione dei pacchetti e il debug del frontend.

- **Mapping e visualizzazione:** per la resa grafica delle mappe e dei percorsi viene utilizzata react-leaflet, permettendo di visualizzare i layer di OpenStreetMap direttamente nel browser.
- **Ambiente di esecuzione:** l'applicazione frontend viene gestita tramite Node.js, utilizzato per la gestione delle dipendenze e l'avvio del server di sviluppo locale.
- **Gestione API:** le chiamate asincrone verso il backend sono affidate ad Axios, garantendo uno scambio dati fluido tra client e server.

Controller. Funge da intermediario tra il Model e la View, gestendo le richieste dell'utente e ottimizzando la logica di interazione:

- **API:** il backend espone endpoint documentati tramite i controller di Spring Boot. Ogni richiesta HTTP inviata dal frontend tramite Axios viene gestita dal metodo specifico.
- **Ottimizzazione:** il controller riceve in input coordinate di partenza e di arrivo, interroga i servizi di routing e restituisce i risultati elaborati, in base al percorso ottimizzato.
- **Formato dati:** lo scambio di informazioni tra client e server avviene esclusivamente in formato JSON.

1.6 Strumenti utilizzati

Di seguito vengono illustrati gli strumenti e le tecnologie adottate per la realizzazione del progetto. La scelta dello stack tecnologico è stata orientata a garantire la massima integrazione tra la gestione dei dati geografici e un'architettura di tipo modulare.

- **Spring Boot:** Framework Java basato sull'ecosistema Spring, impiegato per implementare la logica di backend secondo il pattern architetturale MVC. Spring Boot facilita la gestione del Controller e l'integrazione del Model con i database, automatizzando la configurazione dell'infrastruttura. Questo approccio garantisce una chiara separazione delle responsabilità, rendendo l'applicazione scalabile, modulare e facilmente manutenibile.
- **GraphHopper:** motore di routing utilizzato per ottenere dati reali di distanza e tempo di percorrenza tra le coordinate GPS dei punti di interesse. Fornisce la componente pratica necessaria per calcolare percorsi realistici sulle strade esistenti, garantendo itinerari ottimizzati basati su tempi reali di percorrenza e condizioni stradali.
- **JUnit 4:** Framework per il testing unitario in Java, impiegato per validare il corretto funzionamento delle classi e dei metodi sviluppati. L'uso di test automatizzati consente di rilevare tempestivamente eventuali errori e di garantire la robustezza del codice.
- **MongoDB e MongoDB Compass:** MongoDB è il database NoSQL utilizzato per memorizzare i POI (Points of Interest) con strutture flessibili e attributi variabili provenienti da OpenStreetMap, come nome, descrizione, coordinate, categorie e immagini. Spring Data MongoDB facilita le operazioni CRUD e le query complesse. MongoDB Compass, invece, è lo strumento grafico che permette di esplorare visivamente le collezioni, verificare la correttezza dei documenti JSON e correggere manualmente eventuali dati errati, migliorando il controllo sulla qualità dei dati.
- **OpenStreetMap:** piattaforma open-source che fornisce dati geospaziali dettagliati. I dati di OpenStreetMap sono utilizzati da Nominatim per il geocoding, da Overpass API per interrogazioni sui monumenti e da React-Leaflet per la visualizzazione delle mappe interattive nel frontend. La natura collaborativa di OpenStreetMap garantisce dati aggiornati e accurati per il calcolo dei percorsi.
- **Nominatim:** servizio di geocoding utilizzato per trasformare indirizzi testuali inseriti dall'utente in coordinate geografiche precise (*lat/lon*), consentendo di posizionare correttamente il punto di partenza dei percorsi sulla mappa.

- **Overpass API:** interroga il database OpenStreetMap per individuare monumenti e POI nelle vicinanze del punto di interesse indicato dall'utente. Per evitare di sovraccaricare il servizio e garantire tempi di risposta rapidi, i dati vengono memorizzati localmente su MongoDB dopo la prima richiesta, rendendo più efficienti le ricerche successive.
- **React.js & Node.js:** React gestisce lo stato dell'interfaccia utente, come la lista dei monumenti selezionati e le informazioni sul percorso ottimizzato, mentre Node.js offre l'ambiente di esecuzione e la gestione dei pacchetti tramite npm. Insieme, permettono di costruire un frontend dinamico, reattivo e facilmente aggiornabile.
- **React-Leaflet:** libreria che connette React alle mappe interattive di OpenStreetMap. Consente di posizionare marker sui monumenti, visualizzare linee di percorso ottimizzate calcolate dal backend e rendere l'esperienza dell'utente più intuitiva grazie a mappe dinamiche e interattive.
- **Axios:** libreria per gestire chiamate HTTP asincrone. Permette al frontend di inviare richieste al backend e ricevere le risposte in formato JSON senza ricaricare la pagina, garantendo un'esperienza utente fluida e immediata.
- **Eclipse:** IDE principale per la scrittura del codice Java, il debugging del backend e la gestione del progetto tramite Maven o Gradle. Offre strumenti avanzati di refactoring, analisi del codice e integrazione con sistemi di versionamento, migliorando l'efficienza dello sviluppo.
- **CodeMR:** strumento di analisi statica del codice sorgente, che estrae metriche di coesione e accoppiamento delle classi. Aiuta a identificare classi troppo complesse e suggerisce possibili rifattorizzazioni, migliorando la manutenibilità del software.
- **Postman:** Strumento essenziale per il testing delle API REST sviluppate con Spring Boot. Consente di effettuare richieste HTTP, validare le risposte del server e automatizzare test, facilitando così il debug e l'integrazione tra i diversi componenti del sistema.
- **StarUML & Diagrams.net:** strumenti utilizzati per la modellazione dei sistemi. StarUML serve per diagrammi formali (casi d'uso, classi, deployment), mentre Diagrams.net è utile per schemi logici, flussi di dati e rappresentazioni più informali, favorendo la chiarezza della progettazione.
- **GitHub & GitHub Desktop:** strumenti di controllo di versione che permettono di salvare lo storico dei file, gestire branch multipli e sincronizzare il lavoro tra

diversi membri del team, garantendo una collaborazione efficiente e tracciabilità delle modifiche.

- **WhatsApp:** applicazione di messaggistica utilizzata come strumento di comunicazione interna tra i membri del team. Consente di coordinare le attività di sviluppo, discutere problemi tecnici e organizzare riunioni in tempo reale, garantendo un flusso comunicativo rapido ed efficiente.

Capitolo 2

Iterazione 1

2.1 Casi d'uso implementati

In questa fase sono stati sviluppati i seguenti casi d'uso ritenuti prioritari per lo sviluppo di OptiTour.

ID	Titolo
UC1.1	Registrazione
UC1.2	Login
UC1.3	Logout
UC11	Imposta punto di partenza
UC12.1	Creazione itinerario manuale
UC13	Scelta tempistiche
UC3	Visualizza percorsi
UC16	Elimina viaggio
UC14	Ottimizzazione percorso

Tabella 2.1: Iterazione 1

UC1.1: Registrazione

Attori: Utente, Sistema.

Descrizione: Un nuovo utente può creare un account personale fornendo i dati richiesti.

Flusso degli eventi:

1. L'utente accede alla pagina di registrazione.
2. Inserisce i propri dati personali e le credenziali.
3. Il sistema verifica i dati e crea il nuovo account.
4. L'utente viene reindirizzato alla pagina di login.

UC1.2: Login

Attori: Utente, Sistema.

Descrizione: Un utente registrato può accedere al proprio account inserendo le proprie credenziali.

Flusso degli eventi:

1. L'utente accede alla pagina di login.
2. Inserisce email e password.
3. Il sistema verifica le credenziali e autentica l'utente.
4. L'utente viene reindirizzato alla home.

UC1.3: Logout

Attori: Utente, Sistema.

Descrizione: L'utente può terminare la propria sessione in modo sicuro.

Flusso degli eventi:

1. L'utente clicca sul pulsante di logout.
2. Il sistema termina la sessione corrente.
3. L'utente viene reindirizzato alla pagina di login.

UC11: Imposta punto di partenza

Attori: Utente, Sistema

Descrizione: L'utente inserisce un indirizzo testuale come punto di partenza del viaggio. Il sistema converte automaticamente l'indirizzo in coordinate geografiche tramite il servizio esterno Nominatim e le salva nel viaggio.

Flusso degli eventi:

1. L'utente inserisce un indirizzo di partenza nel campo apposito.
2. Il sistema invia l'indirizzo al servizio Nominatim.
3. Nominatim restituisce le coordinate geografiche corrispondenti.
4. Le coordinate vengono salvate nel viaggio e la mappa si aggiorna sul punto selezionato.

UC12.1: Creazione itinerario manuale

Attori: Utente, Sistema.

Descrizione: L'utente seleziona manualmente i monumenti che desidera visitare. I monumenti vengono recuperati da Overpass API la prima volta e salvati in MongoDB per

evitare chiamate ripetute al servizio esterno.

Flusso degli eventi:

1. L'utente cerca i monumenti di una città.
2. Il sistema verifica se i monumenti sono già presenti in MongoDB.
3. Se non presenti, il sistema li recupera tramite Overpass API e li salva nel database.
4. I monumenti vengono mostrati sulla mappa.
5. L'utente seleziona i monumenti che desidera visitare.
6. L'utente conferma la selezione e il sistema salva il viaggio.

UC13: Scelta tempistiche

Attori: Utente, Sistema.

Descrizione: L'utente può definire per ciascun monumento selezionato il tempo che intende dedicare alla visita, espresso in minuti.

Flusso degli eventi:

1. L'utente seleziona un monumento durante la creazione del viaggio.
2. Inserisce il tempo di visita desiderato in minuti.
3. Il sistema salva il tempo di visita associato a quella tappa del viaggio.

UC3.2: Visualizza percorsi

Attori: Utente, Sistema.

Descrizione: L'utente può visualizzare i propri viaggi e di visualizzare i dettagli del singolo viaggio.

Flusso degli eventi:

1. L'utente accede alla sezione dei propri viaggi.
2. Il sistema recupera tutti i viaggi dell'utente dal database.
3. L'utente può selezionare un singolo viaggio per visualizzarne i dettagli.

UC16: Elimina viaggio

Attori: Utente, Sistema.

Descrizione: L'utente può eliminare un viaggio esistente.

Flusso degli eventi:

1. L'utente seleziona il viaggio che desidera eliminare.
2. Conferma l'operazione di eliminazione.
3. Il sistema rimuove il viaggio dal database.

UC14: Ottimizzazione percorso

Attori: Utente, Sistema

Descrizione: Il sistema ottimizza l'ordine delle tappe del viaggio, calcolando il percorso più efficiente tra i monumenti selezionati a partire dal punto di partenza definito dall'utente.

Flusso degli eventi:

1. Il sistema recupera il viaggio tramite l'ID.
2. Legge le coordinate del punto di partenza e di ogni monumento dal database.
3. Applica l'algoritmo di ottimizzazione per calcolare il percorso ottimale.
4. Il viaggio viene aggiornato con il nuovo ordine delle tappe e lo stato viene impostato a SAVED.

2.2 Diagramma delle Interfacce

Il diagramma in Figura 2.1 mostra le interfacce del sistema con le relative funzioni operative e i dati di input e output. La rappresentazione evidenzia inoltre la netta separazione dei livelli applicativi (layer) in conformità al design pattern architetturale Model-View-Controller (MVC).

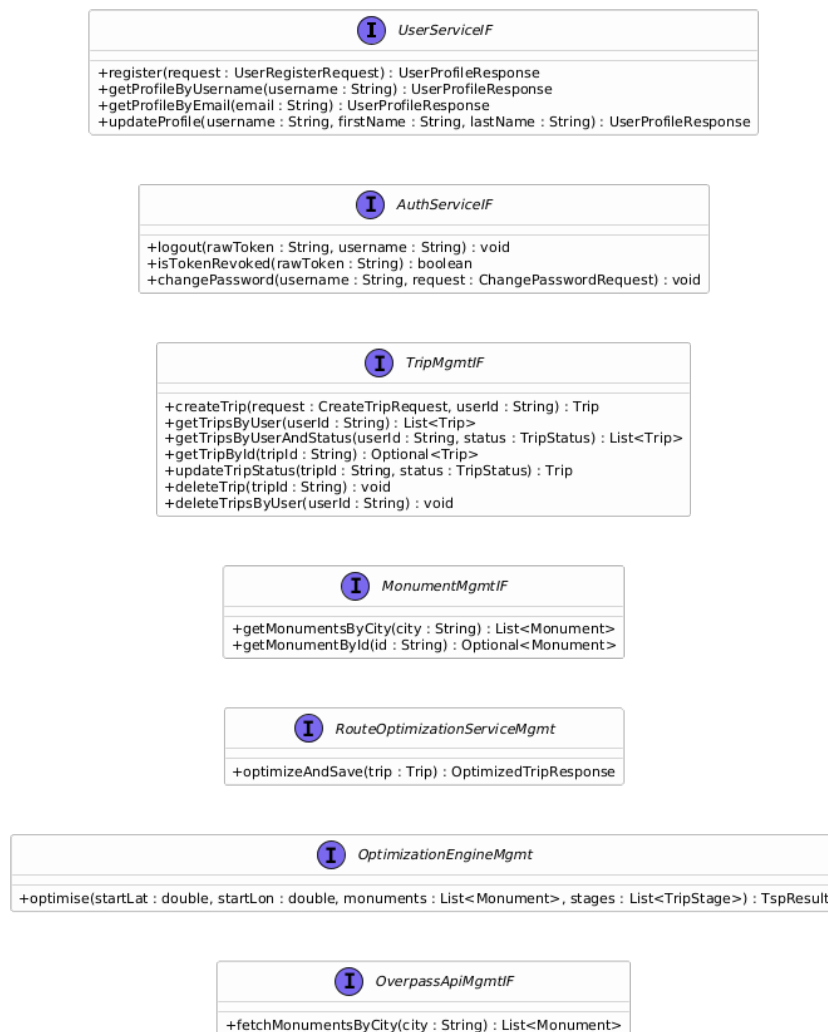


Figura 2.1: Diagramma delle interfacce

2.3 UML Class Diagram

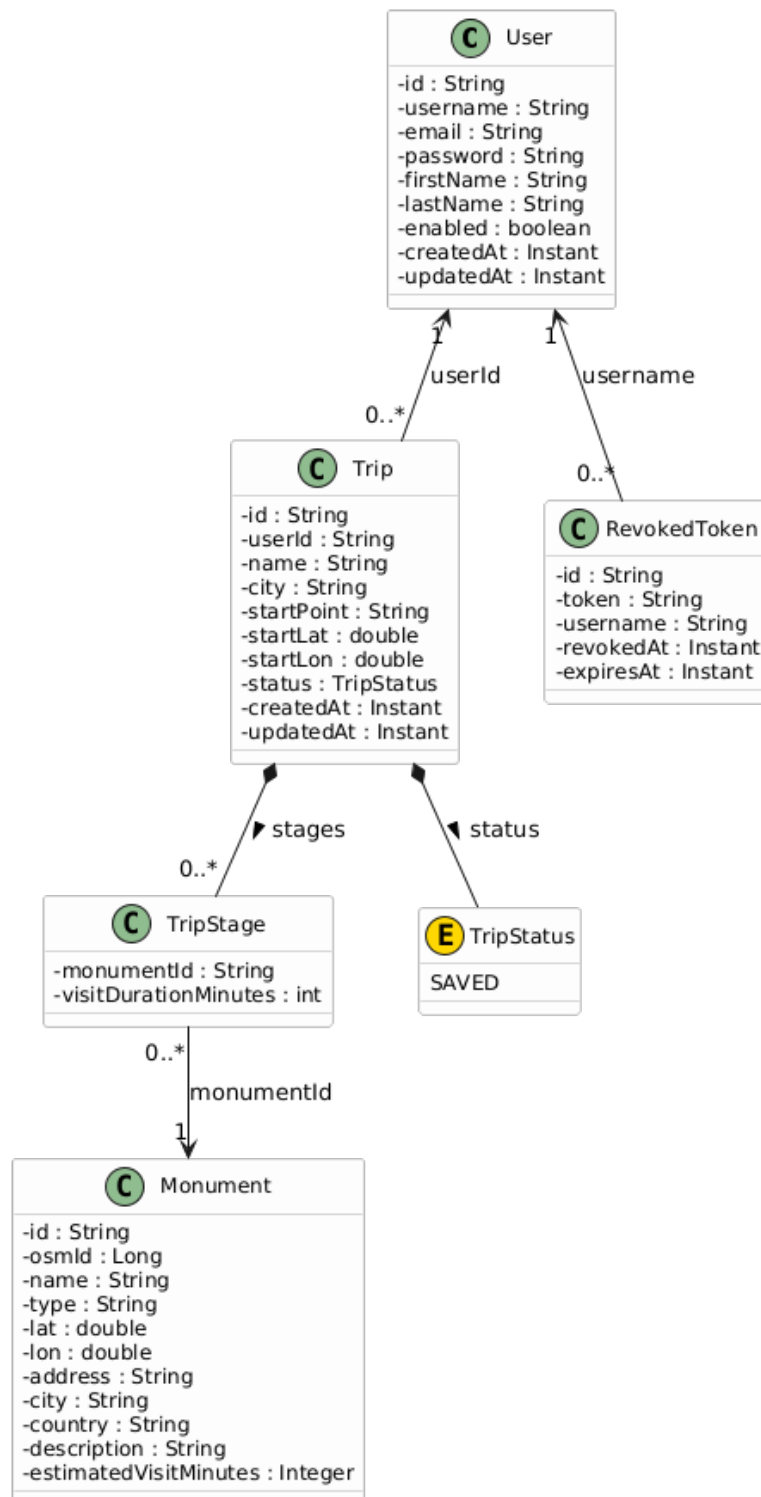


Figura 2.2: Diagramma delle classi

2.4 Component Diagram

A partire dai casi d'uso, è stata progettata l'architettura del sistema seguendo un'organizzazione a più livelli, al fine di separare chiaramente interfaccia utente, logica applicativa e persistenza dei dati. Questa suddivisione consente di ottenere un sistema modulare ed estendibile.

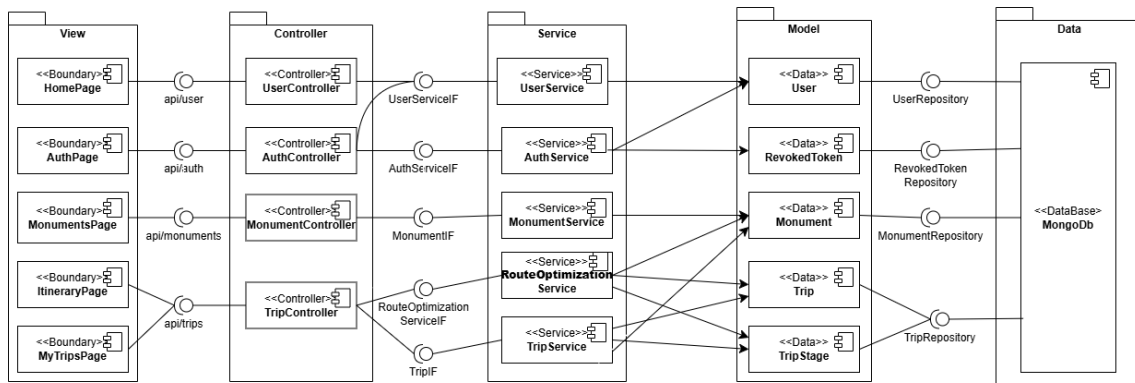


Figura 2.3: Diagramma dei componenti

Il diagramma è strutturato in cinque livelli principali: *View*, *Controller*, *Service*, *Model* e *Data*.

- **View:** rappresenta i componenti *boundary* del front-end, ovvero le pagine con cui l'utente interagisce direttamente: *HomePage*, *AuthPage*, *MonumentsPage*, *ItineraryPage* e *MyTripsPage*. Questi componenti comunicano con il backend attraverso le API esposte dai controller.
- **Controller:** contiene i componenti che ricevono le richieste HTTP e le indirizzano verso la logica applicativa appropriata. Il sistema espone endpoint distinti per la gestione degli utenti, dell'autenticazione, dei monumenti e dei viaggi: *UserController*, *AuthController*, *MonumentController* e *TripController*.
- **Service:** include la logica di business del sistema. I controller delegano a questi componenti l'esecuzione delle operazioni principali attraverso interfacce dedicate, quali *UserServiceIF*, *AuthServiceIF*, *MonumentIF*, *RouteOptimizationServiceIF* e *TripIF*. Le rispettive implementazioni, ovvero *UserService*, *AuthService*, *MonumentService*, *RouteOptimizationService* e *TripService*, si occupano di gestire i dati e le funzionalità del sistema. In particolare, la gestione dei viaggi distingue tra gestione del viaggio e ottimizzazione del percorso.

- **Model:** definisce le entità del dominio, che rappresentano gli oggetti principali gestiti dall'applicazione: *User*, *RevokedToken*, *Monument*, *Trip* e *TripStage*. Queste classi costituiscono il modello dei dati utilizzato dai service e dal database.
- **Data:** comprende i componenti responsabili dell'accesso al database, ovvero i repository *UserRepository*, *RevokedTokenRepository*, *MonumentRepository* e *TripRepository*, tutti collegati al database *MongoDb*. Questo livello gestisce le operazioni di lettura e scrittura, mantenendo separata la logica di accesso ai dati dalla logica applicativa.

Nel complesso, il diagramma presenta una struttura coerente con il principio di separazione delle responsabilità: le componenti di *View* interagiscono con i *Controller*, i *Controller* delegano ai *Service* la logica di business, i *Service* utilizzano le entità del *Model* e accedono ai dati tramite il livello *Data*. Questa organizzazione rende l'architettura chiara e robusta.

2.5 Deployment Diagram

Il seguente deployment diagram rappresenta la distribuzione fisica dei componenti software dell'applicazione all'interno dei diversi nodi di esecuzione.

Il sistema è suddiviso in quattro nodi principali: *device*, *server*, *database* ed *external system*. Ogni nodo contiene degli artefatti (file o pacchetti di codice che vengono deployati) e componenti di runtime.

Questi nodi interagiscono tra loro mediante opportuni protocolli di comunicazione.

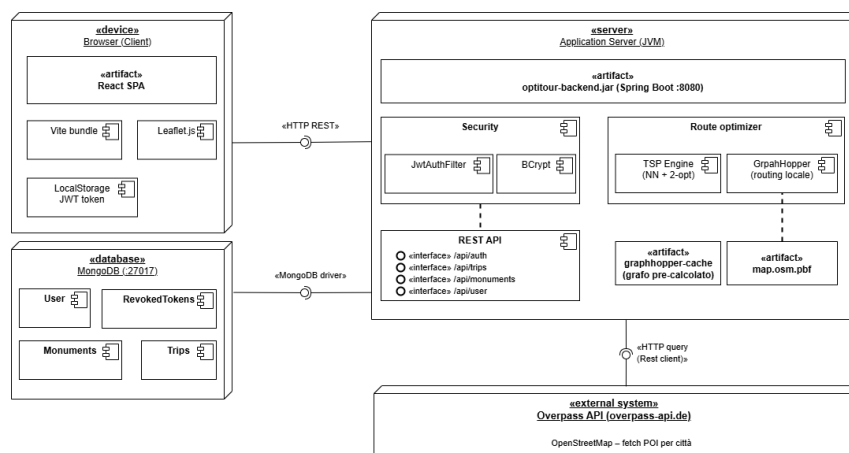


Figura 2.4: Diagramma di deployment

- **Nodo «device» → Browser(client):** rappresenta il dispositivo dell'utente finale e contiene l'artefatto: **React SPA** che a sua volta è composto dal *Vite bundle* e dalla libreria *Leaflet.js* per la visualizzazione delle mappe.

Il browser utilizza inoltre *localStorage* per memorizzare i JWT token necessari per l'autenticazione delle richieste verso il backend.

La comunicazione verso il nodo server avviene tramite **HTTP REST** dove il token viene inserito nella header *Authorization*.

- **Nodo «server» → Application Server(JVM):** rappresenta l'ambiente di esecuzione del backend basato su **Spring Boot** e distribuito come:

«artifact»: **optitour-backend.jar**: il quale viene esposto sulla porta :8080.

All'interno del nodo sono presenti i seguenti componenti:

- **Sicurezza:** intercetta e valida le richieste mediante *JwtAuthFilter* e si occupa dell'hashing delle password tramite *BCrypt*.
- **REST API** → espone tramite gli endpoint: */api/auth*, */api/trips*, */api/monuments* e */api/user*.
- **Motore di ottimizzazione:**

- * TPS Engine (Nearest neighbor + 2-opt).
- * GraphHopper per il routing locale.

– **Artefatti aggiuntivi**

- * **«artifact»map.osm.pbf:** dataset OpenStreetMap.
- * **«artifact»graphhopper-cache:** grafo pre-calcolato per il routing.

Il nodo server comunica con MongoDB tramite **Spring Data MongoDB** e con Overpass API tramite **HTTP query (REST client)**.

- **Nodo «database» → MongoDB:** rappresenta il database NOSQL utilizzato per la persistenza dei dati dell'applicazione.
È esposto sulla porta :27017 e comunica con il backend tramite il driver MongoDB integrato in SpringData.
- **Nodo «eternal system » → Overpass API:** rappresenta il sistema esterno basato su OpenStreetMap che viene utilizzato per il recupero dei POI (Point Of Interest) relativi ad una città. Il backend interagisce con esso mediante richieste **HTTP**, utilizzando un client REST.

2.6 Ottimizzazione del percorso

Dato un insieme di monumenti e un punto di partenza, si cerca un percorso chiuso minimo che visiti tutti i monumenti una sola volta e torni al punto di partenza. L'algoritmo implementato combina due fasi principali:

1. **Heuristica Nearest Neighbour:** costruisce un tour iniziale partendo dal nodo di partenza e visitando il nodo più vicino non ancora visitato.
2. **Ottimizzazione 2-opt:** migliora il tour iniziale invertendo coppie di archi quando ciò riduce la distanza totale.

Pseudocodice

Algorithm 1 Costruzione matrice delle distanze

```

1: function BUILDDISTANCEMATRIX(startPoint, monuments)
2:   nodes  $\leftarrow$  [startPoint] + monuments, size  $\leftarrow$  numero di nodi
3:   dist  $\leftarrow$  matrice vuota di dimensione size  $\times$  size
4:   for i = 0 to size-1 do
5:     for j = 0 to size-1 do
6:       if i = j then
7:         dist[i][j]  $\leftarrow$  0
8:       else
9:         dist[i][j]  $\leftarrow$  distanza(nodes[i], nodes[j])
10:      end if
11:    end for
12:  end for
13:  return dist
14: end function

```

Algorithm 2 Nearest Neighbour

```

1: function NEARESTNEIGHBOUR(dist)
2:   size  $\leftarrow$  numero di nodi, visited  $\leftarrow$  array[size] = false
3:   tour[0]  $\leftarrow$  0, visited[0]  $\leftarrow$  true
4:   for step = 1 to size-1 do
5:     current  $\leftarrow$  tour[step-1], bestNext  $\leftarrow$  nodo più vicino non visitato
6:     tour[step]  $\leftarrow$  bestNext, visited[bestNext]  $\leftarrow$  true
7:   end for
8:   return tour
9: end function

```

Algorithm 3 2-opt

```

1: function TWOOPT(tour, dist)
2:   improved  $\leftarrow$  true
3:   while improved do
4:     improved  $\leftarrow$  false
5:     for i = 0 to size-2 do
6:       for j = i+2 to size-1 do
7:         if i = 0 and j = size-1 then continue
8:         end if
9:         if scambio riduce distanza then
10:           inverti sottosequenza da i+1 a j, improved  $\leftarrow$  true
11:         end if
12:       end for
13:     end for
14:   end while
15:   return tour
16: end function

```

Analisi della complessità

Nella fase di costruzione della matrice delle distanze, l'algoritmo impiega due cicli annidati per calcolare i percorsi tra tutti i nodi, risultando in una complessità temporale pari a $O(n^2)$, dove n rappresenta il numero totale di nodi (il punto di partenza unito ai monumenti da visitare). Successivamente, l'euristica costruttiva *Nearest Neighbour* valuta, nel caso peggiore, tutti i nodi rimanenti a ogni passo per individuare la destinazione più vicina. Anche in questo caso, la ricerca porta a una complessità temporale di $O(n^2)$. Infine, nella fase di ottimizzazione locale tramite l'algoritmo *2-opt*, vengono confrontate iterativamente coppie di archi nel tentativo di ridurre il costo totale del percorso. Nel caso peggiore, i cicli annidati necessari per queste valutazioni e le relative operazioni di scambio portano a una complessità pari a $O(n^3)$. Di conseguenza, la complessità asintotica totale dell'intero algoritmo è dominata da quest'ultima fase, attestandosi su $O(n^3)$.

2.7 Test

In questa sezione vengono descritte le attività di testing, finalizzate alla verifica della qualità e dell’affidabilità del sistema.

2.7.1 Test: CodeMR

L’analisi statica conferma che OptiTour presenta una struttura solida. Il software risulta pulito e privo di componenti critici. L’assenza di “classi problematiche” dimostra che il codice è stato sviluppato con cura, mantenendo ogni modulo isolato e facilmente gestibile. Questo garantisce che il sistema sia semplice da mantenere e pronto per le iterazioni successive del progetto.

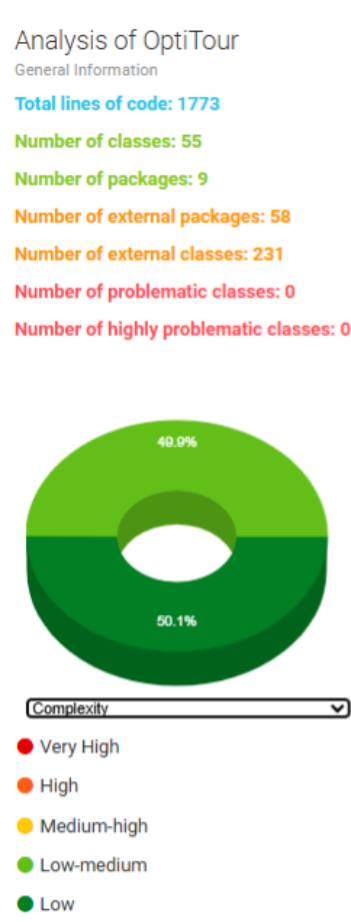


Figura 2.5: Analisi della complessità.

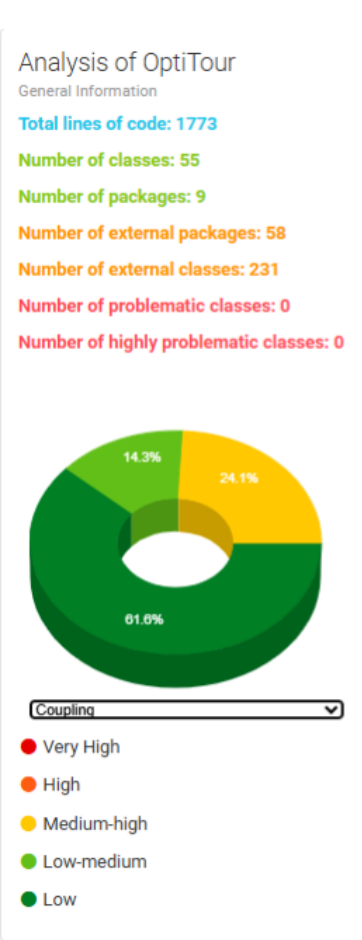


Figura 2.6: Grado di accoppiamento.

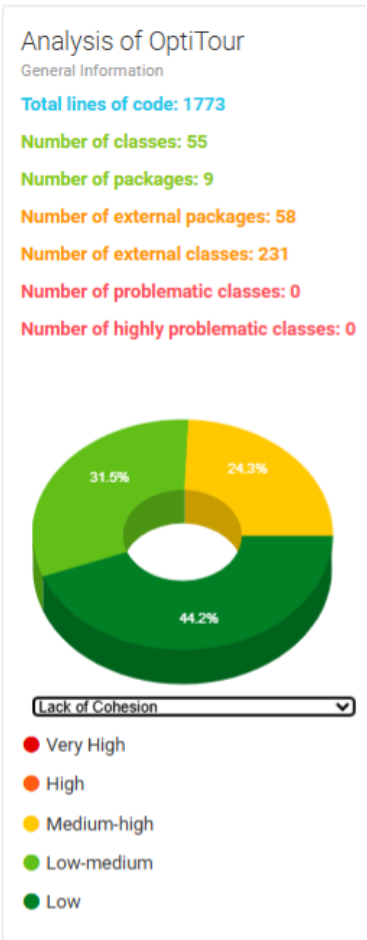


Figura 2.7: Livello di coesione.

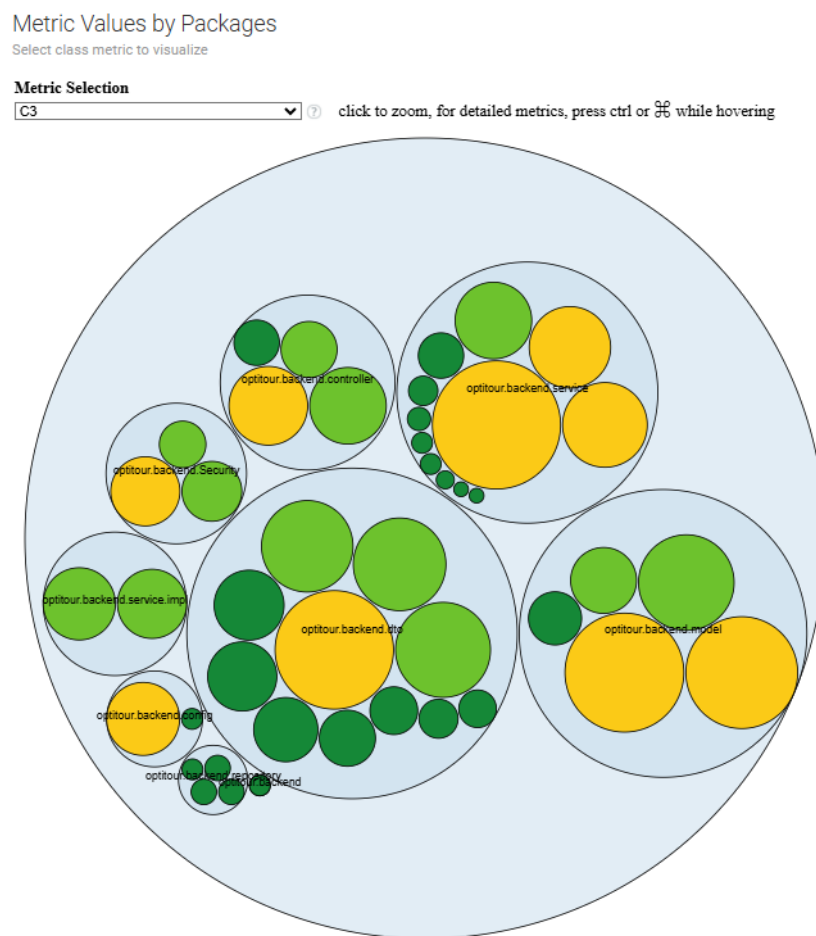


Figura 2.8: Distribuzione dei componenti: organizzazione delle classi nei package Model, View/DTO, Service e Controller.

2.7.2 Test: Junit

Per garantire l'affidabilità del sistema e la correttezza della logica di business, è stata implementata una suite di test automatizzati. Di seguito vengono riportati gli esiti dei test eseguiti per le diverse componenti.

1. Autenticazione e Gestione Utenti

I test relativi all'autenticazione coprono i flussi di registrazione, login (con generazione del token JWT) e gestione del profilo. Come si può osservare nelle Figura 2.9 e Figura 2.10, sono stati verificati sia gli endpoint che la logica di business sottostante.

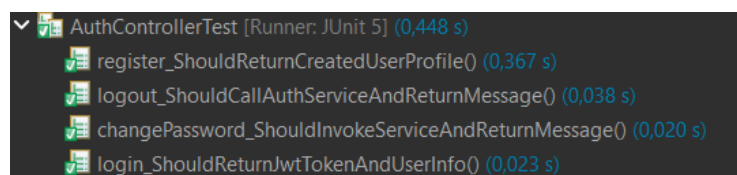


Figura 2.9: Test degli endpoint di autenticazione.

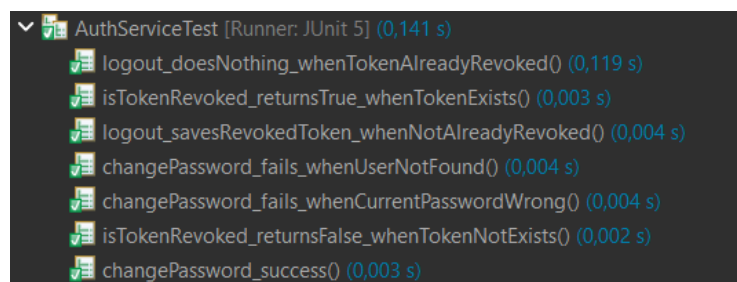


Figura 2.10: Test della logica di autenticazione.

Allo stesso modo, le Figura 2.11 e Figura 2.12 mostrano il superamento dei test relativi alla gestione anagrafica degli utenti.

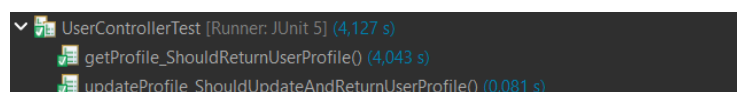


Figura 2.11: Test degli endpoint profilo utente.

2. Gestione Monumenti

I test verificano il corretto recupero dei dati, validando sia l'integrità delle risposte (Figura 2.13) sia il meccanismo di salvataggio locale dopo la chiamata a Overpass API (Figura 2.14).

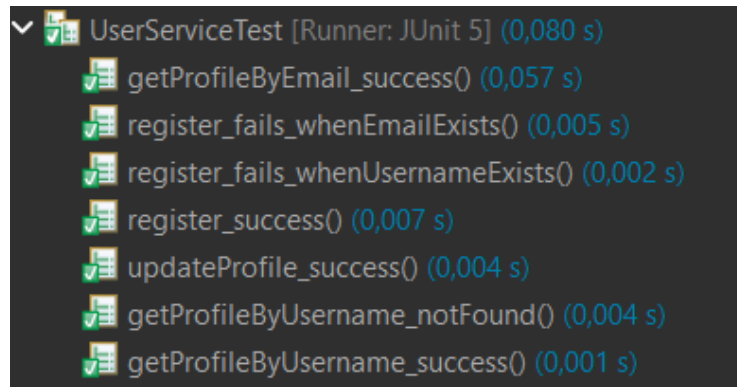


Figura 2.12: Test della logica gestione utente.

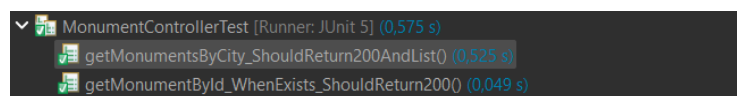


Figura 2.13: Test degli endpoint monumenti.

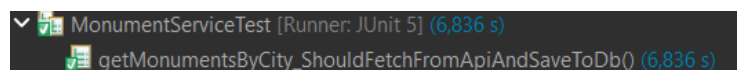


Figura 2.14: Test integrazione Overpass API.

3. Motore di Ottimizzazione e Viaggi

Questa sezione rappresenta il cuore algoritmico del sistema. La Figura 2.17 illustra la validazione del TSP, mentre nelle Figura 2.18 e Figura 2.15 vengono testate l'integrazione con Nominatim e la persistenza finale dei viaggi ottimizzati. Inoltre, la Figura 2.16 mostra il superamento dei test degli endpoint relativi ai viaggi.

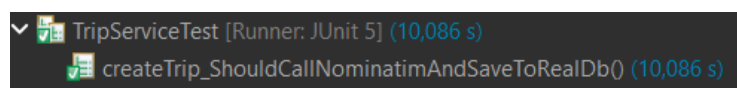


Figura 2.15: Test creazione viaggi con Nominatim.

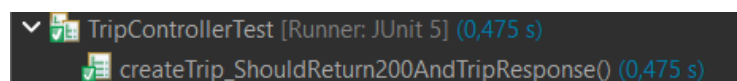


Figura 2.16: Test degli endpoint relativi ai viaggi.

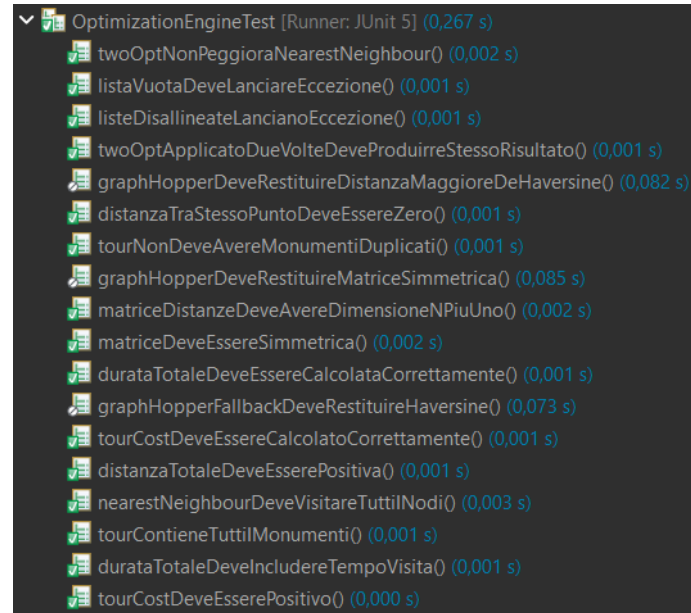


Figura 2.17: Validazione approfondita dell’algoritmo di ottimizzazione TSP.

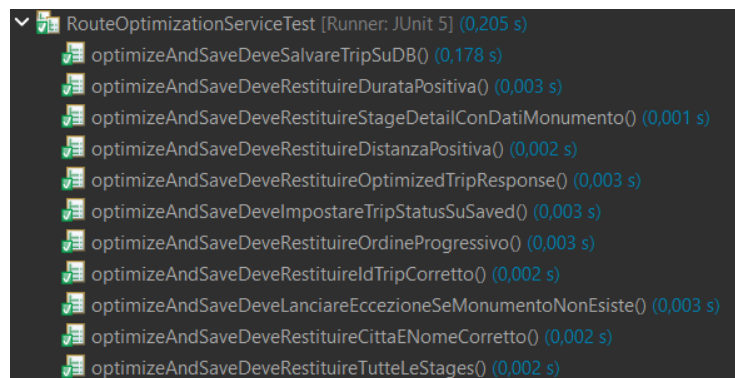


Figura 2.18: Test del servizio di ottimizzazione.

2.7.3 API Esposte

In questa sezione vengono descritte le principali API esposte dal backend tramite i controller di Spring Boot, necessarie per supportare i casi d'uso sviluppati in questa prima fase e testate con successo tramite Postman.

1. Autenticazione e Profilo Utente

Registrazione Utente

- **Endpoint:** POST /api/auth/register
- **Descrizione:** Registra un nuovo utente nel sistema creando un account personale (come mostrato in Figura 2.19).
- **Payload:**
 - username (string) - L'username scelto dall'utente.
 - email (string) - L'indirizzo email dell'utente.
 - password (string) - La password dell'account.

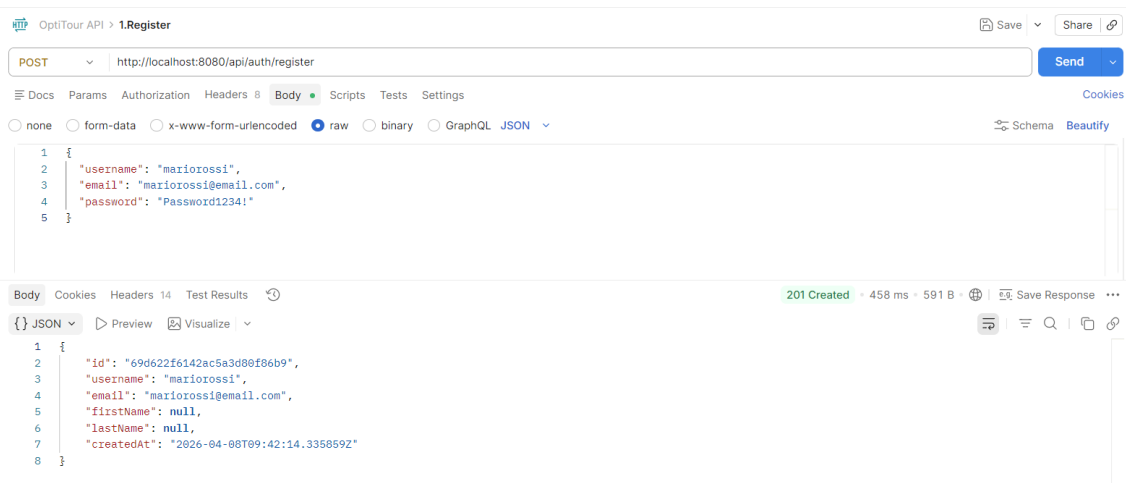


Figura 2.19: Test su Postman: Registrazione di un nuovo utente nel sistema.

Autenticazione (Login)

- **Endpoint:** POST /api/auth/login
- **Descrizione:** Autentica l'utente validando le credenziali e restituisce il token JWT di sessione (vedi Figura 2.20).
- **Payload:**

- `usernameOrEmail` (string) - L'username o l'email dell'utente.
- `password` (string) - La password dell'account.

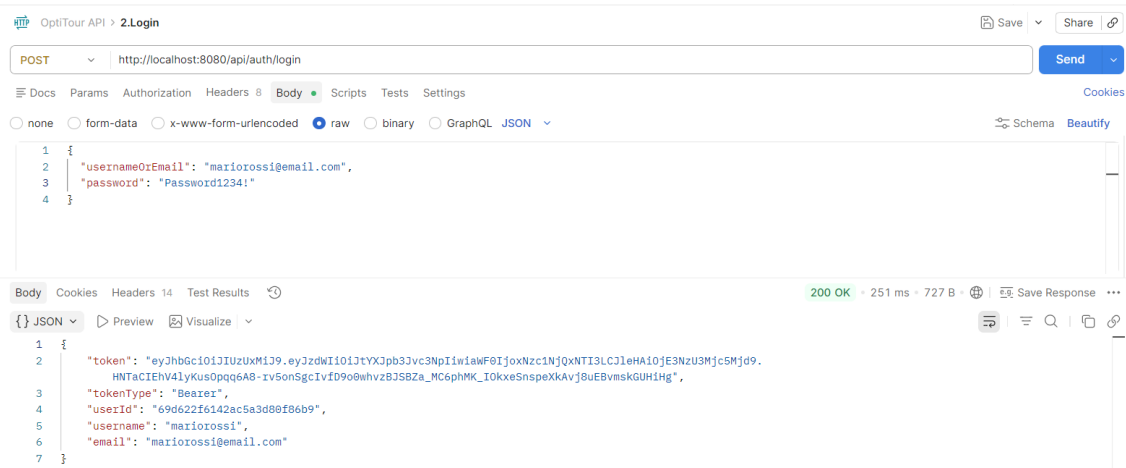


Figura 2.20: Test su Postman: Login utente e generazione del token di autenticazione JWT.

2. Gestione Monumenti

Ricerca Monumenti per Città

- **Endpoint:** GET `/api/monuments`
- **Descrizione:** Recupera la lista dei monumenti per una specifica città (vedi Figura 2.21). Se non presenti nel DB, innesca la chiamata verso Overpass API.
- **Parametri:**
 - `city` (string) - Query parameter con il nome della città.

3. Gestione Viaggi e Ottimizzazione

Creazione di un Viaggio

- **Endpoint:** POST `/api/trips`
- **Descrizione:** Crea un nuovo itinerario manuale associato all'utente, salvando punto di partenza, tappe e stime temporali di visita. I dettagli della richiesta e della risposta sono illustrati in Figura 2.22 e Figura 2.23.

GET `http://localhost:8080/api/monuments?city=Bergamo`

200 OK · 48 ms · 75.4 KB

	id	name	type	lat	lon	address	city	country	description
0	69d62484142ac5a3d80f86ba	Giuseppe Garibaldi	memorial	45.815023	10.0734976	null	Bergamo	null	null
1	69d62484142ac5a3d80f86bb	Vittorio Emanuele II	memorial	45.8163915	10.0739154	null	Bergamo	null	null
2	69d62484142ac5a3d80f86bc	Arboreto Alpino Gleno	attraction	45.9976133	10.0798595	null	Bergamo	null	null
3	69d62484142ac5a3d80f86bd	Museo dell'Ottocento	museum	45.7040517	9.6662563	null	Bergamo	null	null
4	69d62484142ac5a3d80f86be	Museo Civico di Scienze Naturali "E. Caffi"	museum	45.7057438	9.659708	Piazza della Cittadella	Bergamo	null	null
5	69d62484142ac5a3d80f86bf	Museo Civico Archeologico	museum	45.7059394	9.6598246	Piazza della Cittadella	Bergamo	null	null
6	69d62484142ac5a3d80f86c0	Porta delle Cornacchie	viewpoint	45.9605327	9.7358984	null	Bergamo	null	null
7	69d62484142ac5a3d80f86c1	Monte Foldone	viewpoint	45.8491307	9.6166591	null	Bergamo	null	Il Monte Foldone presenta due sommità: la Cima Nord, che è di pochi metri più bassa ma presenta un cartello sulla cima e la Cima Sud, meno conosciuta ma che in realtà sarebbe la vera cima, di 7 metri più alta.
8	69d62484142ac5a3d80f86c2	Pizzo Rabbioso	viewpoint	45.8379826	9.7034663	null	Bergamo	null	null
9	69d62484142ac5a3d80f86c3	Monte Gioco	viewpoint	45.8572235	9.7028919	null	Bergamo	null	null
10	69d62484142ac5a3d80f86c4	Monte Sparavera	viewpoint	45.8080587	9.9542616	null	Bergamo	null	null
11	69d62484142ac5a3d80f86c5	Colle Tomenone	viewpoint	45.6728932	9.7819235	null	Bergamo	null	null

Figura 2.21: Test su Postman: Recupero della lista dei monumenti per la città di Bergamo.

• Parametri e Payload:

- **userId** (string) - Query parameter con l'ID dell'utente.
- **name** (string) - Nome assegnato al viaggio.
- **city** (string) - Città del viaggio.
- **startPoint** (string) - Punto di partenza.
- **stages** (array) - Lista delle tappe (monumenti) selezionate.

POST `http://localhost:8080/api/trips?userId=69d622f6142ac5a3d80f86b9`

Send

Schema Beautify

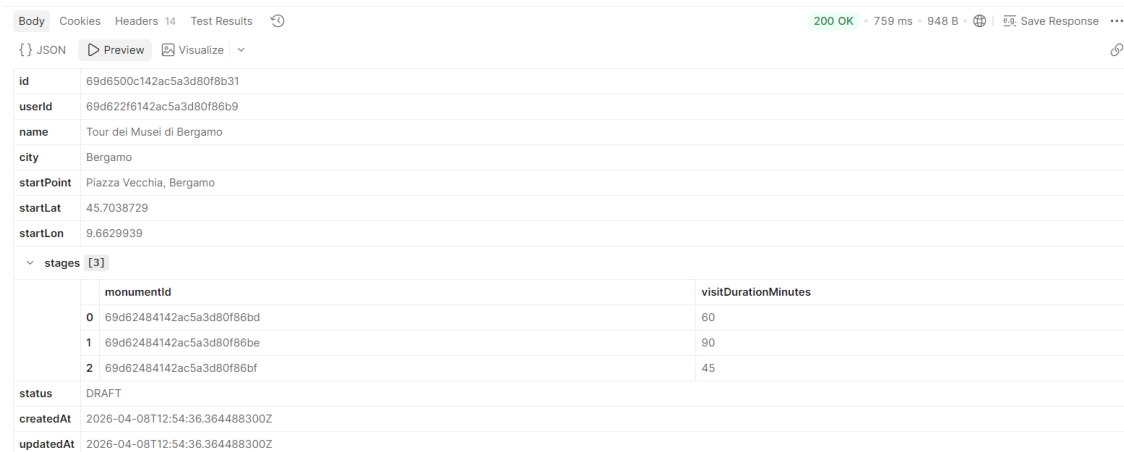
```

1 {
2   "name": "Tour dei Musei di Bergamo",
3   "city": "Bergamo",
4   "startPoint": "Piazza Vecchia, Bergamo",
5   "stages": [
6     {
7       "monumentId": "69d62484142ac5a3d80f86bd",
8       "visitDurationMinutes": 60
9     },
10    {
11      "monumentId": "69d62484142ac5a3d80f86be",
12      "visitDurationMinutes": 90
13    },
14    {
15      "monumentId": "69d62484142ac5a3d80f86bf",
16      "visitDurationMinutes": 45
17    }
18  ]
19 }

```

Figura 2.22: Test su Postman: Payload di richiesta per la creazione di un nuovo viaggio.

Ottimizzazione Percorso



Body		
id	69d6500c142ac5a3d80f8b31	
userId	69d622f6142ac5a3d80f8b69	
name	Tour dei Musei di Bergamo	
city	Bergamo	
startPoint	Piazza Vecchia, Bergamo	
startLat	45.7038729	
startLon	9.6629939	
stages	[3]	
	monumentId	visitDurationMinutes
0	69d62484142ac5a3d80f86bd	60
1	69d62484142ac5a3d80f86be	90
2	69d62484142ac5a3d80f86bf	45
status	DRAFT	
createdAt	2026-04-08T12:54:36.364488300Z	
updatedAt	2026-04-08T12:54:36.364488300Z	

Figura 2.23: Test su Postman: Risposta del server con il salvataggio del viaggio.

- **Endpoint:** POST `/api/trips/{id}/optimize`
- **Descrizione:** Avvia l'algoritmo di ottimizzazione (TSP) sulle tappe. Restituisce il nuovo ordine calcolato aggiornando lo stato in **SAVED**, come evidenziato in Figura 2.24.
- **Parametri:**
 - `{id}` (string) - ID del viaggio da ottimizzare.

OptiTour API

>

5. ottimizza

Save

Share

POST

http://localhost:8080/api/trips/69d652a7142ac5a3d80f8b32/optimize

Send

Docs

Params

Authorization

Headers

Body

Scripts

Tests

Settings

Body

Cookies

Headers

14

Test Results

{}

JSON

Preview

Visualize

200 OK

- 255 ms

- 1.2 KB

Save Response

tripId

69d652a7142ac5a3d80f8b32

tripName

Tour dei Musei di Bergamo

city

Bergamo

startLat

45.7038729

startLon

9.6629939

stages

[3]

	order	monumentId	name	type	lat	lon	address	visitDurationMinutes
0	1	69d62484142ac5a3d80f86bd	Museo dell'Ottocento	museum	45.7040517	9.6662563	null	60
1	2	69d62484142ac5a3d80f86bf	Museo Civico Archeologico	museum	45.7059394	9.6598246	Piazza della Cittadella	45
2	3	69d62484142ac5a3d80f86be	Museo Civico di Scienze Naturali "E. Caffi"	museum	45.7057438	9.659708	Piazza della Cittadella	90

totalDistanceMeters

1148.6574087766126

totalDurationSeconds

12527

Figura 2.24: Test su Postman: Esecuzione dell'algoritmo di ottimizzazione TSP. La risposta mostra l'itinerario riordinato con calcolo di distanza e tempi totali.

Eliminazione di un Viaggio

- **Endpoint:** DELETE `/api/trips/{id}`

- **Descrizione:** Permette l'eliminazione di un itinerario dal database (vedi Figura 2.25).
- **Parametri:**
 - {id} (string) - ID del viaggio da rimuovere.



Figura 2.25: Test su Postman: Eliminazione corretta di un viaggio con risposta 204 No Content.

Capitolo 3

Iterazione 2

3.1 Casi d'uso implementati

In questa seconda fase di sviluppo, l'attenzione si è concentrata sull'estensione delle funzionalità di gestione dei viaggi, con particolare riguardo alla condivisione e alla modifica granulare degli itinerari. Sono stati implementati il catalogo pubblico (con la relativa funzione di pubblicazione), lo storico, la gestione dei percorsi salvati/preferiti e la possibilità di eliminare singole tappe. Di seguito sono riportati i casi d'uso sviluppati per questa iterazione, in conformità con i requisiti iniziali.

ID	Titolo
UC3.1	Visualizza catalogo
UC3.3	Visualizza storico
UC4	Filtri ricerca
UC5	Salva percorso nei preferiti
UC7	Rimuovi percorso salvato
UC8	Pubblica percorso nel catalogo
UC10	Imposta percorso completato
UC12.2	Creazione itinerario casuale
UC15	Modifica tappe
UC16	Elimina tappa

Tabella 3.1: Iterazione 2 - Casi d'uso implementati

UC3.1: Visualizza catalogo

Attori: Utente, Sistema.

Descrizione: L'utente può visualizzare percorsi creati da altri utenti e resi pubblici,

raccolti all'interno di un catalogo condiviso.

Flusso degli eventi:

1. L'utente accede alla sezione "Catalogo" tramite l'interfaccia principale.
2. Il sistema recupera dal database la lista dei viaggi con visibilità pubblica.
3. Il sistema mostra all'utente l'elenco dei viaggi disponibili.
4. L'utente può selezionare un singolo viaggio per visualizzarne i dettagli.

UC3.3: Visualizza storico

Attori: Utente, Sistema.

Descrizione: L'utente può consultare l'elenco dei propri viaggi passati, contrassegnati come completati.

Flusso degli eventi:

1. L'utente accede alla sezione "Storico" dal proprio profilo.
2. Il sistema interroga il database filtrando i viaggi dell'utente con stato COMPLETED.
3. Il sistema restituisce e mostra a schermo l'elenco dei percorsi effettuati.

UC4: Filtri ricerca

Attori: Utente, Sistema.

Descrizione: L'utente può filtrare i percorsi nel catalogo in base a criteri quali città, durata massima o numero di tappe.

Flusso degli eventi:

1. L'utente imposta i parametri del filtro nella sezione Catalogo.
2. L'utente avvia la ricerca.
3. Il sistema applica i filtri tramite una query su MongoDB.
4. La vista viene aggiornata mostrando solo i risultati pertinenti.

UC5: Salva percorso nei preferiti

Attori: Utente, Sistema.

Descrizione: L'utente può aggiungere un percorso di interesse alla propria sezione dei preferiti.

Flusso degli eventi:

1. L'utente visualizza un viaggio dal Catalogo o dai propri viaggi.
2. Seleziona l'opzione di salvataggio nei preferiti.
3. Il sistema crea l'associazione nel database.
4. Il sistema conferma l'avvenuto salvataggio.

UC7: Rimuovi percorso salvato

Attori: Utente, Sistema.

Descrizione: L'utente può rimuovere un itinerario precedentemente salvato nella propria area personale.

Flusso degli eventi:

1. L'utente accede alla sezione dei percorsi salvati.
2. Seleziona il viaggio da rimuovere.
3. Il sistema elimina l'associazione nel database.
4. La lista viene aggiornata rimuovendo l'elemento dall'interfaccia.

UC8: Pubblica percorso nel catalogo

Attori: Utente, Sistema.

Descrizione: L'utente rende pubblico un proprio viaggio, permettendo ad altri utenti di visualizzarlo nel catalogo.

Flusso degli eventi:

1. L'utente apre i dettagli di un proprio viaggio salvato.
2. Seleziona l'opzione "Pubblica nel catalogo".
3. Il sistema aggiorna lo stato di visibilità dell'itinerario nel database.
4. Il percorso diventa visibile a tutti gli utenti nella sezione Catalogo.

UC10: Imposta percorso completato

Attori: Utente, Sistema.

Descrizione: L'utente contrassegna un viaggio come completato al termine della visita.

Flusso degli eventi:

1. L'utente seleziona un viaggio attivo.
2. Clicca su "Segna come completato".
3. Il sistema aggiorna lo stato in COMPLETED.
4. Il viaggio viene spostato automaticamente nella sezione Storico.

UC12.2: Creazione itinerario casuale

Attori: Utente, Sistema.

Descrizione: Generazione automatica di un itinerario basato su parametri casuali (città e numero tappe).

Flusso degli eventi:

1. L'utente richiede un viaggio casuale specificando la città.
2. Il sistema recupera i POI (da database o Overpass API).
3. Il sistema seleziona un set casuale di monumenti rispettando i vincoli.
4. Viene generato e mostrato il percorso ottimizzato sulla mappa.

UC15: Modifica tappe

Attori: Utente, Sistema.

Descrizione: L'utente modifica un viaggio esistente aggiungendo nuovi monumenti e richiedendo il ricalcolo dell'itinerario.

Flusso degli eventi:

1. L'utente apre un viaggio in modalità modifica.
2. Aggiunge una o più tappe selezionandole dalla mappa o dalla lista.
3. Il sistema invoca l'Optimization Engine per ricalcolare il percorso TSP.
4. Il viaggio aggiornato viene salvato con i nuovi dati di tempo e distanza.

UC16: Elimina tappa

Attori: Utente, Sistema.

Descrizione: L'utente può rimuovere una singola tappa da un itinerario esistente.

Flusso degli eventi:

1. L'utente apre i dettagli di un viaggio salvato.
2. Identifica la tappa da rimuovere e seleziona l'opzione di eliminazione.
3. Il sistema rimuove la tappa dalla lista e ricalcola immediatamente l'ottimizzazione del percorso rimanente.
4. L'interfaccia aggiorna la mappa e i dettagli del viaggio.

3.2 Diagramma delle interfacce



Figura 3.1: Diagramma delle interfacce - Iterazione 2.

Il diagramma in Figura 3.1 illustra le interfacce aggiornate del sistema per la seconda iterazione. Rispetto alla fase precedente, sono state ampliate le firme dei metodi

nei service, in particolare all'interno di TripMgmtIF, per supportare nativamente le nuove funzionalità legate al recupero dei viaggi pubblici (catalogo), alla gestione dello storico, dei preferiti e all'aggiornamento di un viaggio con la conseguente rimozione/aggiunta di tappe e modifica di durata di visita.

3.3 UML Class Diagram

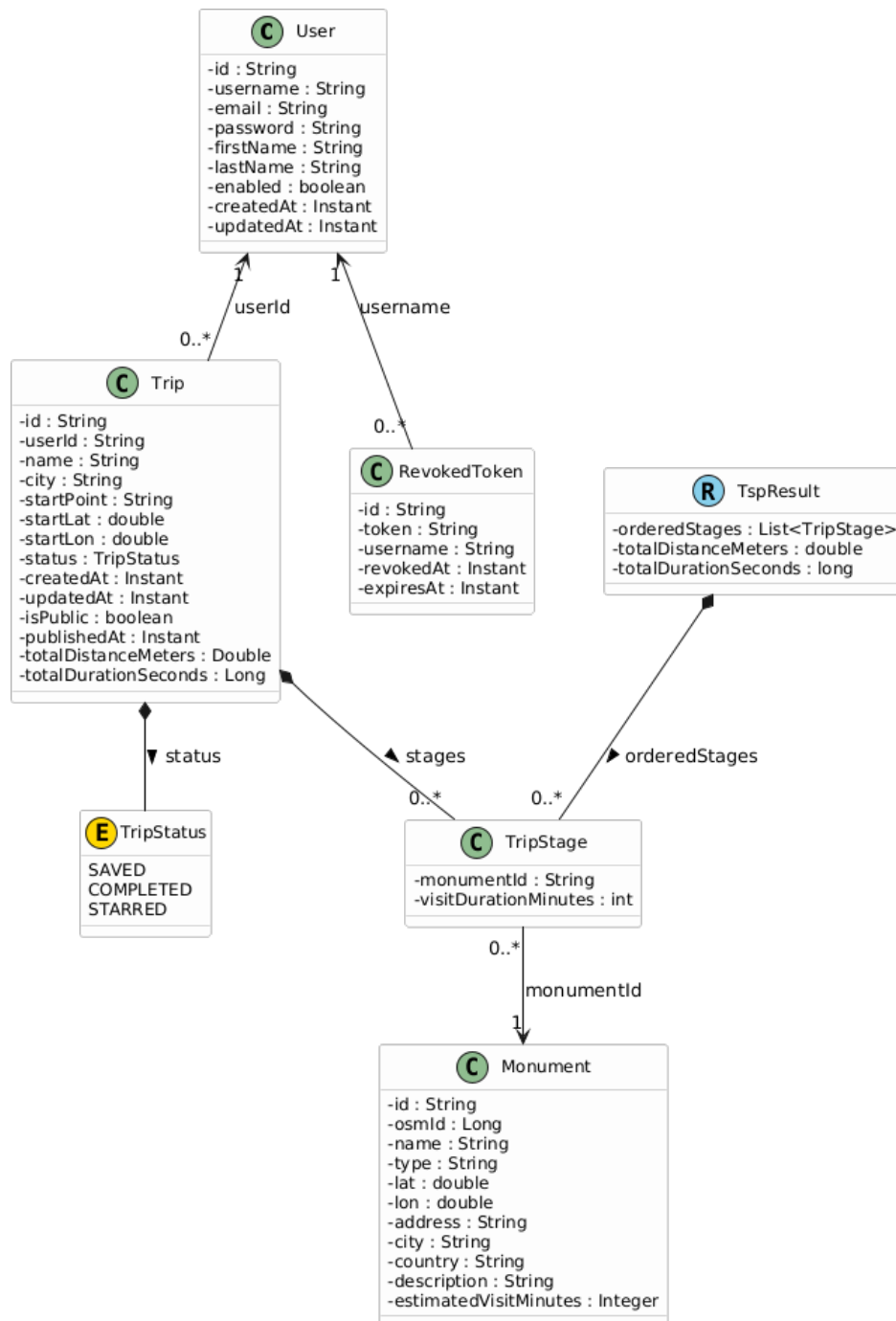


Figura 3.2: UML Class Diagram - Iterazione 2.

Il diagramma delle classi (Figura 3.2) mostra l'evoluzione del modello dati nella seconda iterazione. L'entità principale **Trip** è stata arricchita per poter gestire il nuovo ciclo di vita dei percorsi:

- L'enumerazione `TripStatus` ora include i nuovi stati `COMPLETED` (per lo storico) e `STARRED` (per i preferiti), oltre a `SAVED` già introdotto nella fase precedente.
- Sono stati aggiunti gli attributi `isPublic` e `publishedAt` per gestire la visibilità dei viaggi all'interno del catalogo condiviso.

3.4 Component Diagram

Rispetto alla precedente iterazione, l'architettura del sistema è stata estesa mantenendo invariata la struttura a più livelli già adottata. In questo modo è stata conservata la separazione tra interfaccia utente, logica applicativa e gestione dei dati, introducendo però nuove funzionalità lato front-end e ampliando i componenti esistenti.

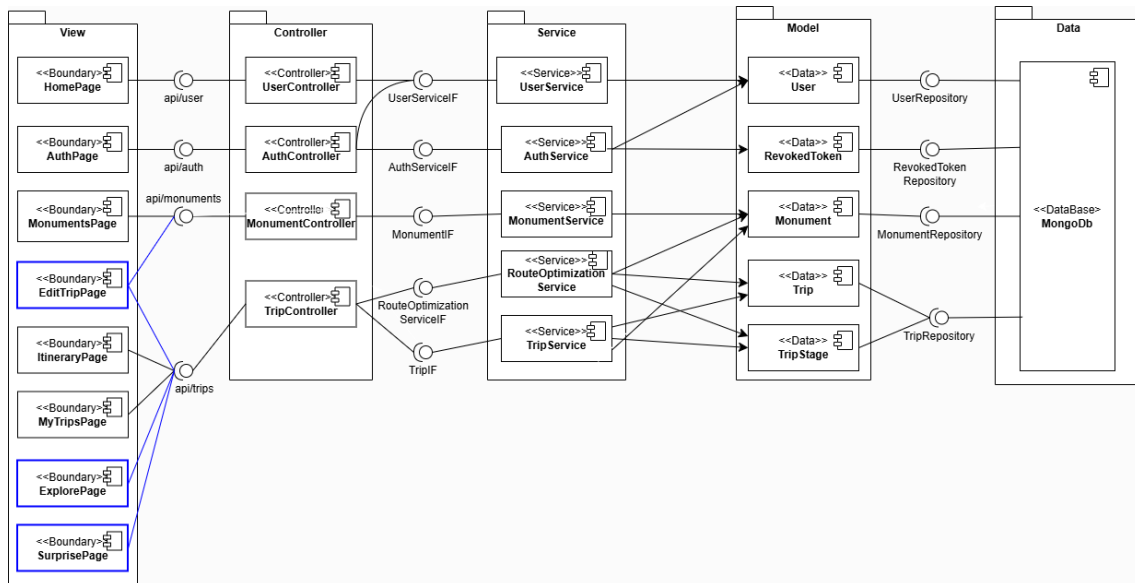


Figura 3.3: Diagramma dei componenti

In questa iterazione sono state aggiunte tre nuove pagine nel livello *View*, ovvero *ExplorePage*, *SurprisePage* ed *EditTripPage*. Gli altri livelli non introducono nuove classi, ma sono stati ampliati nelle funzionalità interne per supportare i nuovi casi d'uso e le nuove interazioni dell'applicazione.

Il diagramma mostra quindi un'evoluzione coerente dell'architettura: le nuove pagine del front-end sfruttano i controller esistenti, che delegano ai service aggiornati, i quali operano sulle entità del *Model* e accedono ai dati attraverso il livello *Data*.

3.5 Deployment diagram

3.6 Test

In questa sezione vengono descritte le attività di testing svolte durante questa iterazione, con l'obiettivo di verificare la correttezza, l'affidabilità e la stabilità delle nuove funzionalità introdotte. I test sono stati condotti sia a livello di singoli componenti, per validare la logica di business, sia a livello di integrazione, per assicurare il corretto funzionamento delle API esposte e la comunicazione tra frontend e backend. Particolare attenzione è stata dedicata alla verifica degli endpoint relativi alla gestione dei viaggi e delle nuove funzionalità introdotte, includendo operazioni di visualizzazione del catalogo pubblico, consultazione dello storico, applicazione dei filtri di ricerca, gestione dei percorsi preferiti e aggiornamento dello stato dei viaggi come completati. Sono state inoltre testate le funzionalità di creazione di itinerari casuali e di modifica delle tappe, comprensive del ricalcolo del percorso ottimizzato tramite il motore TSP.

3.6.1 Test: CodeMR

L'analisi statica del codice per la seconda iterazione è stata condotta tramite CodeMR. Rispetto all'iterazione precedente, le metriche riflettono la naturale espansione del sistema: le linee di codice totali sono aumentate e il numero di classi anche. Nonostante l'introduzione di numerose funzionalità (catalogo pubblico, storico, filtri e percorsi casuali), l'architettura mantiene un livello di qualità elevato, registrando zero classi "altamente problematiche" e solo tre classi "problematiche". Di seguito vengono analizzate le tre metriche principali: Complessità, Accoppiamento e Mancanza di Coesione.

- **Complessità:** L'aggiunta di logiche avanzate ha comportato un fisiologico aumento della complessità in alcune classi. Tuttavia, la maggior parte del sistema si mantiene su livelli bassi o medio-bassi.
- **Accoppiamento:** I risultati confermano l'efficacia dell'architettura a livelli (MVC) e dell'uso delle interfacce. Oltre il 54% delle classi ha un livello di accoppiamento "Basso" e non si registrano classi nella fascia "Alta" o "Molto Alta".
- **Coesione:** La distribuzione rimane bilanciata. Una buona porzione del sistema mostra un'alta coesione (indicata dai valori bassi/verdi di "Lack of Cohesion"), sinonimo di classi robuste, con responsabilità singole ben definite e facili da mantenere.

Analisi delle Classi Complesse e Struttura dei Package

L'analisi di dettaglio ha evidenziato che la classe con il maggior grado di complessità è il `TripService` (Figura 3.7). Questo risultato è ampiamente giustificato: come dimostrato

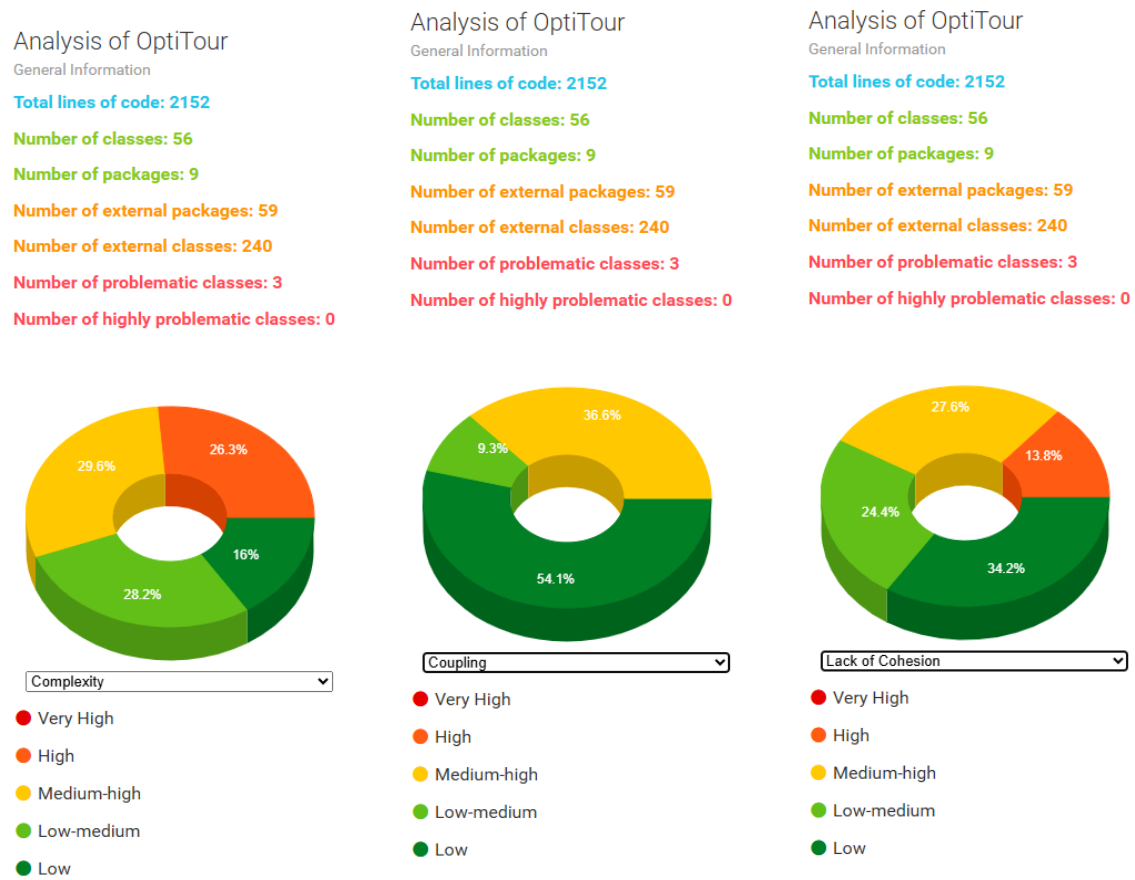


Figura 3.4: Complessità.

Figura 3.5: Accoppiamento.

Figura 3.6: Lack of Cohesion.

Classes with high complexity (#1)

ID	CLASS	COUPLING	COMPLEXITY	LACK OF COHESION	SIZE	LOC	WMC	RFC	NOM	NOP	DT
1	TripService	■	■	■	■	267	57	153	24	4	1

Figura 3.7: Dettaglio della classe TripService, identificata con complessità alta.

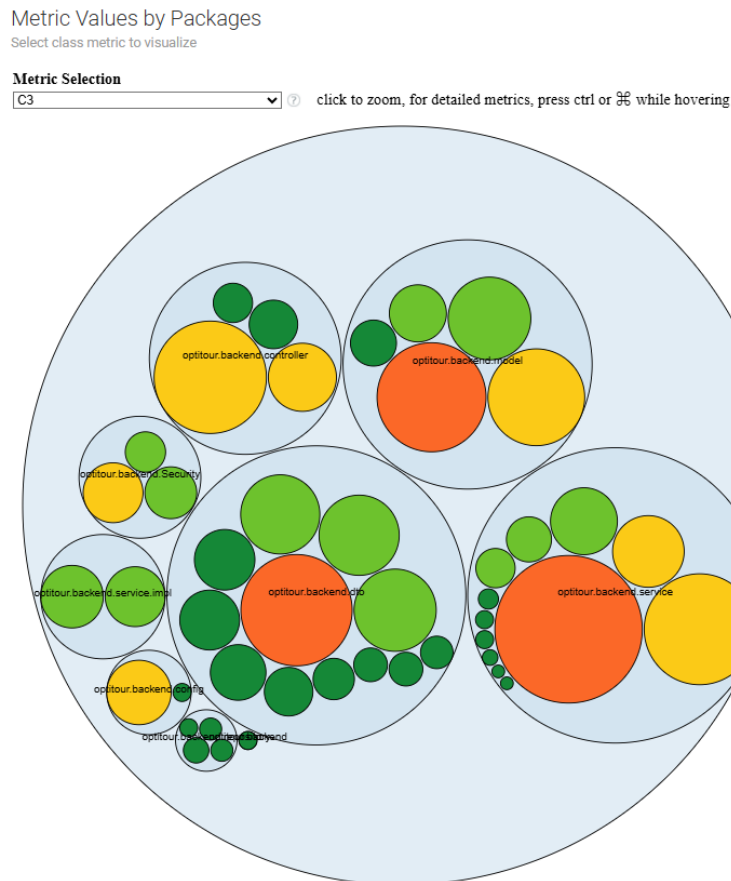


Figura 3.8: Distribuzione delle metriche all'interno dei package architetturali.

anche nei test JUnit, il `TripService` funge da gestore centrale della logica di business, gestendo le validazioni, la sicurezza, la storicizzazione, il ricalcolo del TSP e l'integrazione con i database per tutto il ciclo di vita dei viaggi. L'accoppiamento e la mancanza di coesione per questa classe rimangono comunque nella fascia media. Infine, la visualizzazione della metrica strutturale per package (Figura 3.8) conferma che i pesi maggiori in termini di dimensioni e complessità risiedono nei layer `service`, `model` e `dto`. Al contrario, il layer `controller` mantiene dimensioni più contenute, limitandosi correttamente a intercettare e instradare le chiamate esterne senza inglobare logica applicativa.

3.6.2 Test: Junit

Questa sezione è dedicata ai test eseguiti con JUnit. La copertura dei test riguardanti l'autenticazione base, la gestione degli utenti, l'integrazione con le API dei monumenti e l'algoritmo centrale del TSP è rimasta inalterata rispetto all'Iterazione 1, risultando quindi già del tutto validata.

1. Sicurezza e Validazione Token

Sono stati aggiunti test specifici per il livello di sicurezza, questi nuovi moduli verificano la corretta autorizzazione delle richieste HTTP tramite i filtri JWT, gestendo casi limite come l'uso di token malformati, revocati o scaduti (Figura 3.9 e Figura 3.10), oltre a validare la logica di estrazione dei dettagli utente dal database (Figura 3.11).

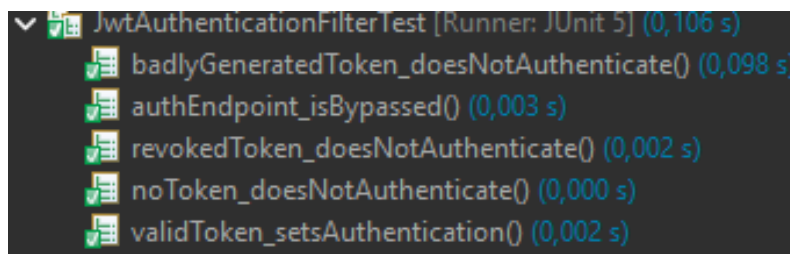


Figura 3.9: Test del filtro di autenticazione JWT e gestione dei token.

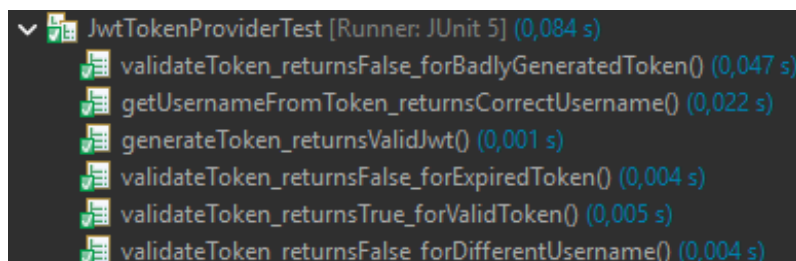


Figura 3.10: Validazione del Provider JWT.

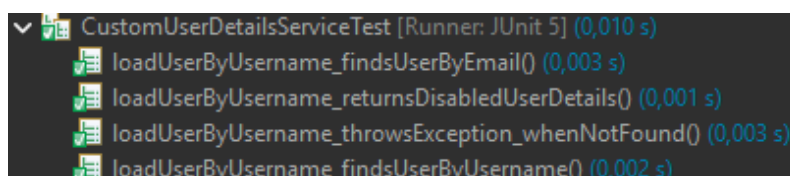


Figura 3.11: Test del Custom User Details Service.

2. Gestione Viaggi (TripService e TripController)

Il cuore del lavoro di testing di questa iterazione si è concentrato sull'espansione dei controller e dei service legati ai viaggi, che ora coprono un vasto ventaglio di nuovi

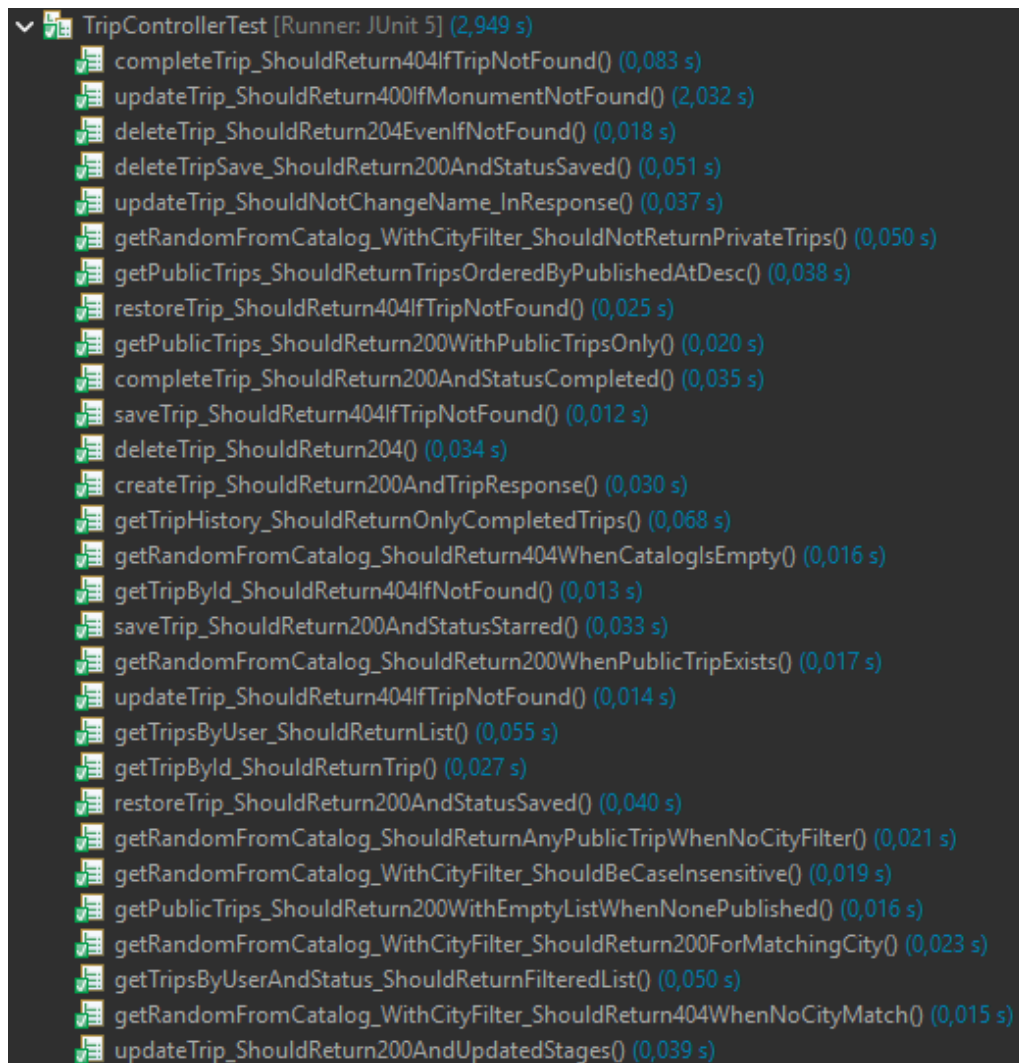
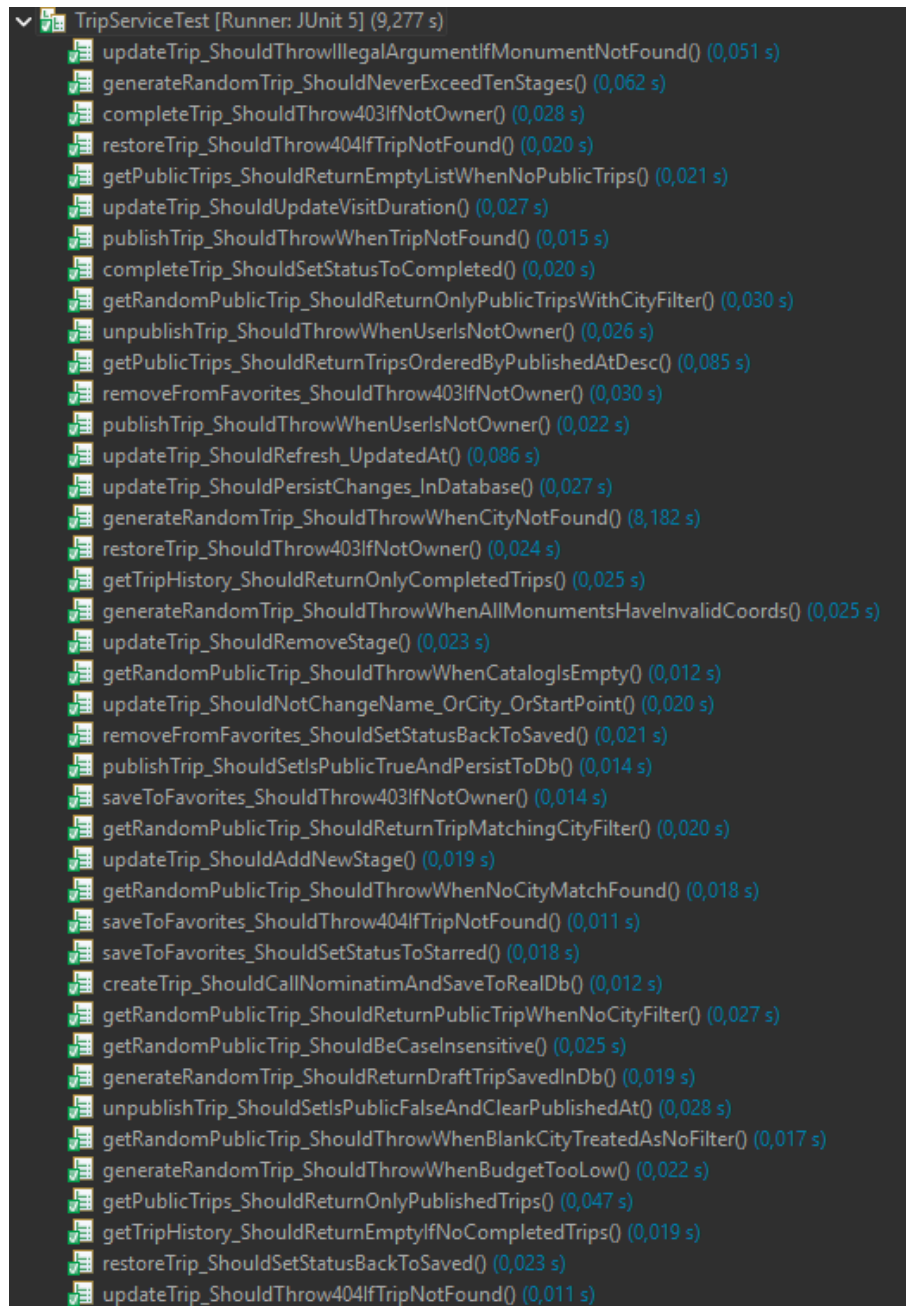


Figura 3.12: Espansione dei test sugli endpoint del TripController.

casi d'uso. Come viene mostrato in Figura 3.12, i test a livello di Controller validano l'esposizione delle nuove API per il recupero dei viaggi pubblici (catalogo), l'applicazione di filtri case-insensitive per città e la corretta gestione degli stati del viaggio (salvataggio nei preferiti, completamento, ripristino).

A livello di logica di business, il `TripServiceTest` (Figura 3.13) è stato arricchito per validare rigorosamente le regole applicative. I test assicurano che:

- La generazione casuale dei viaggi rispetti i vincoli imposti.
- Vengano applicati rigorosi controlli di sicurezza (es. eccezione 403 Forbidden se un utente tenta di modificare o pubblicare un viaggio non suo).
- I flussi di pubblicazione (IsPublic) e la storicizzazione funzionino in stretta sinergia con il database.

Figura 3.13: Espansione dei test della logica di business nel `TripServiceTest`.

3.6.3 API Esposte

In questa sezione vengono descritte le principali API esposte dal backend tramite i controller di Spring Boot, necessarie per supportare i casi d'uso sviluppati e testate tramite Postman.

1. Gestione Viaggi

Aggiornamento di un Viaggio

- **Endpoint:** PUT /api/trips/{id}
- **Descrizione:** Permette di aggiornare un viaggio esistente modificandone le informazioni principali, come la durata per ciascuna tappa, l'aggiunta e la rimozione di tappe con le relative tempistiche. I dettagli della richiesta e della risposta sono illustrati in Figura 3.14 e Figura 3.15.
- **Parametri:**
 - {id} (string) - ID del viaggio da aggiornare.
- **Payload:**
 - name (string) - Nome del viaggio.
 - city (string) - Città del viaggio.
 - startPoint (string) - Punto di partenza.
 - stages (array) - Lista aggiornata delle tappe del viaggio.

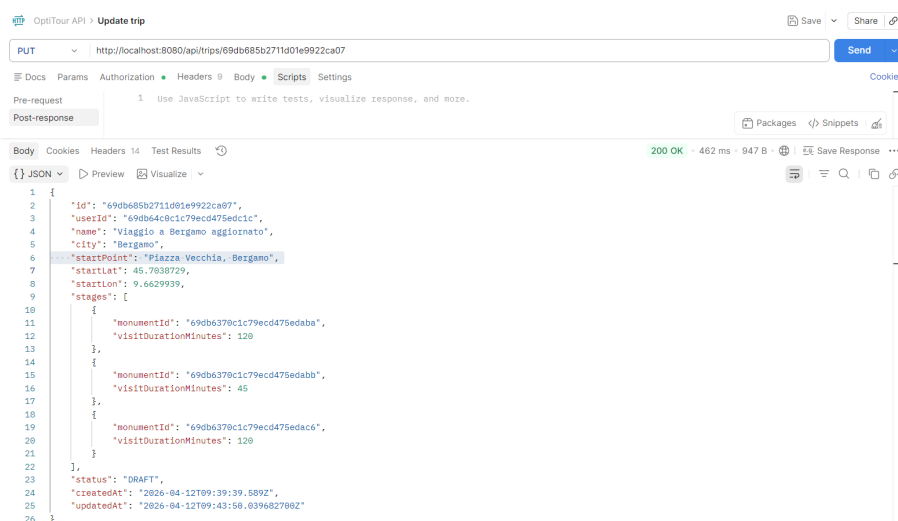


Figura 3.14: Test su Postman: Payload di richiesta per l'aggiornamento di un viaggio.

Body

Cookies

Headers

14

Test Results

200 OK

462 ms

947 B



Save Response

...

{ }

JSON

▶

Preview



Visualize

▼

id

69db685b2711d01e9922ca07

userId

69db64c0c1c79ecd475edc1c

name

Viaggio a Bergamo aggiornato

city

Bergamo

startPoint

Piazza Vecchia, Bergamo

startLat

45.7038729

startLon

9.6629939

▼

stages

[3]

	monumentId	visitDurationMinutes
0	69db6370c1c79ecd475edaba	120
1	69db6370c1c79ecd475edabb	45
2	69db6370c1c79ecd475edac6	120

status

DRAFT

createdAt

2026-04-12T09:39:39.589Z

updatedAt

2026-04-12T09:43:50.039682700Z

Figura 3.15: Test su Postman: Risposta del server dopo l’aggiornamento del viaggio.